H. F. Oliveira Rocha, *Practical Event-Driven Microservices Architecture*
https://doi.org/10.1007/978-1-4842-7468-2_4

4. Structural Patterns and Chaining Processes

Hugo Filipe Oliveira Rocha[1]
(1) Ermesinde, Portugal

This chapter covers:

- Understanding why strong consistency and transactions shouldn't be an option in distributed event-driven systems
- Why we should avoid two-phase commit and distributed transactions in distributed systems
- Using orchestration pattern to implement higher-level business processes
- Using choreography pattern as an alternative to orchestration
- Understanding CQRS (command query responsibility segregation) and event sourcing and applying it to an event-driven system
- Optimizing read queries by building multiple read models
- How to live with invisible domain flow and avoid the pitfall of a spaghetti architecture

As we start to learn computer science and software engineering, data consistency is a cornerstone to software development. Strong consistency and ACID guarantees are the very foundation of most applications. We are used to treating data as a single copy; when we write something, it is instantaneously available for every client to see. If a write fails, the data is always kept in a valid state and changes are roll backed. We use transactions to guarantee consistency across several tables, and often they are the foundation to fulfill complex business processes.

In fact, this way of dealing with data is often wired into our minds. There's an awe-inspiring purity in a database schema under the third normal form.[1] The referential integrity and lack of duplication of a well-

structured third normal form schema are a flimsy glimpse of an ideal platonic world. Data and consistency are precious, perishing the thought of having inconsistent or stale data returned to clients. Knowing the application will always fetch the latest state and every write will immediately be available is straightforward to understand, easy to reason with and facilitates developments. We are used to this bubble of safety and comfort ACID provides. And there's nothing wrong with it; actually in simple and small or medium-sized data applications (by medium-sized, I mean data that fits a single machine without overwhelming infrastructure costs), there's arguably little gain in adopting a distributed architecture as we discussed in Chapters 1 and 2.

However, for distributed or large data applications, it is a different reality. These strong consistency properties, easiness of joining information, and guarantees of transactions between disparate domains are often bastions of a dying order. As we walk toward a distributed architecture, we find we no longer live in that safety bubble a single database with ACID guarantees provides. We are faced with the challenges of how to manage processes that span several services with different databases.

In the previous chapters, we mentioned how guaranteeing strong consistency was a challenge in distributed architectures, but we never detailed why. Section 4.1 will discuss why guaranteeing a consistent business process that spans several domains might be challenging and why it is different from a typical single-process application.

We will approach how to manage a business process that needs the intervention of several different domains in Sections 4.2 and 4.3. These kinds of business processes are called Sagas, and in event-driven architectures, we can apply two types of solutions to handle these complicated business processes, either by orchestration or choreography. We will discuss how to apply both of these solutions in these two sections. Section 4.4 will discuss how we can combine both orchestration and choreography and how they can complement each other.

The distributed nature of event-driven architectures implies that the data is distributed throughout several independent services. Obtaining an aggregated view of data that spans several different services can be a

challenge. Sections 4.5 and 4.6 will approach patterns to build denormalized views in event-driven architectures.

4.1 The Challenges of Transactional Consistency in Distributed Systems

Chapter 1 discussed the challenges of distributed event-driven architectures and the importance of an incremental adoption of event-driven services to deal with those challenges sustainably. One crucial consequence of deconstructing a single monolithic database into several smaller ones is the loss of the ability to guarantee consistency across several domains seamlessly.

Let's illustrate this with an example. If we have a single monolithic eCommerce platform, with a single monolithic database handling the user's orders, each order creation will change the relevant tables. Figure 4-1 illustrates this situation with a stock and order table.

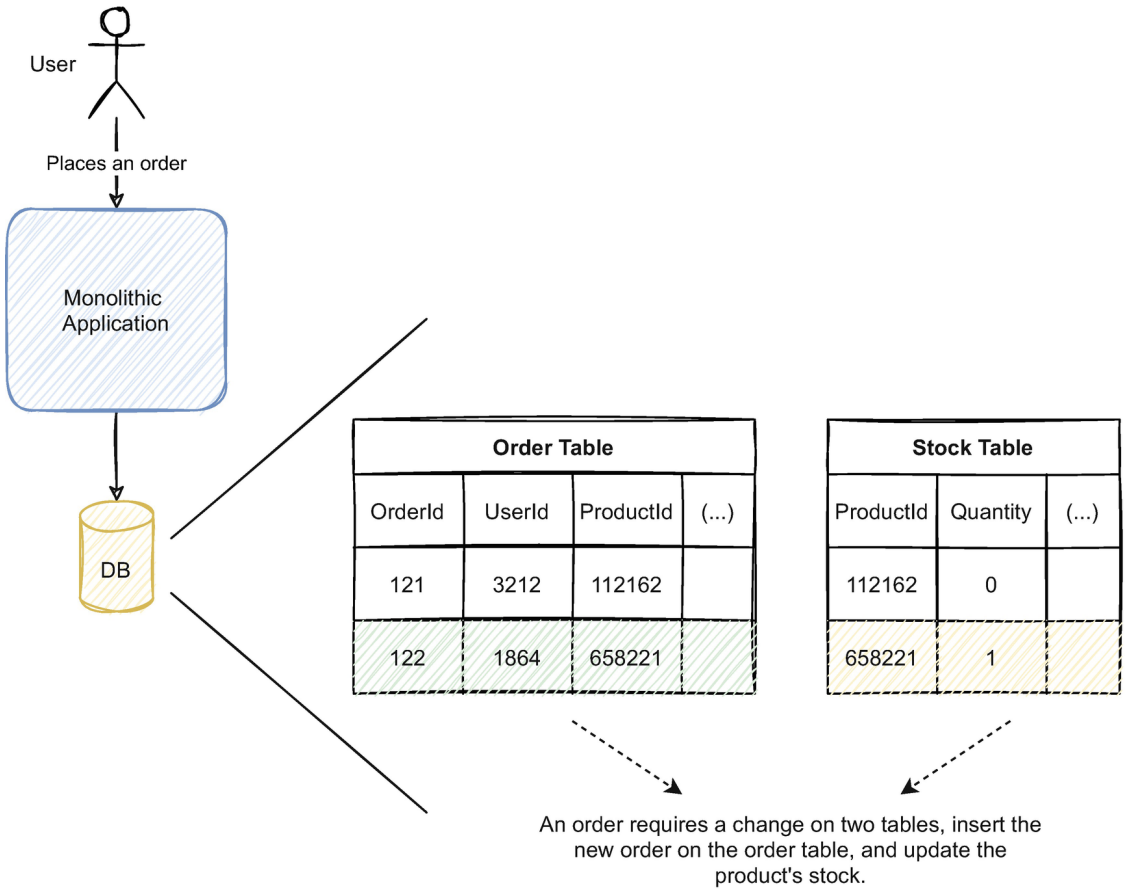

Figure 4-1 The creation of the order inserts a line in the order table and updates the stock quantity in the stock table. Both changes can happen transactionally

When the user requests a new order, the service inserts a new line in the order table, and the stock quantity is updated to reflect that user's order. Since both tables are inside the same database, we can use its ACID properties to guarantee that both changes happen simultaneously or don't happen at all, rejecting the order. Once the changes are made, they will also be immediately available to every client requesting them.

By moving to a distributed event-driven architecture, we would deconstruct the monolithic application into several services. We would also divide the monolithic database and distribute the data between them. Figure 4-2 illustrates how the same example could look like in an event-driven architecture.

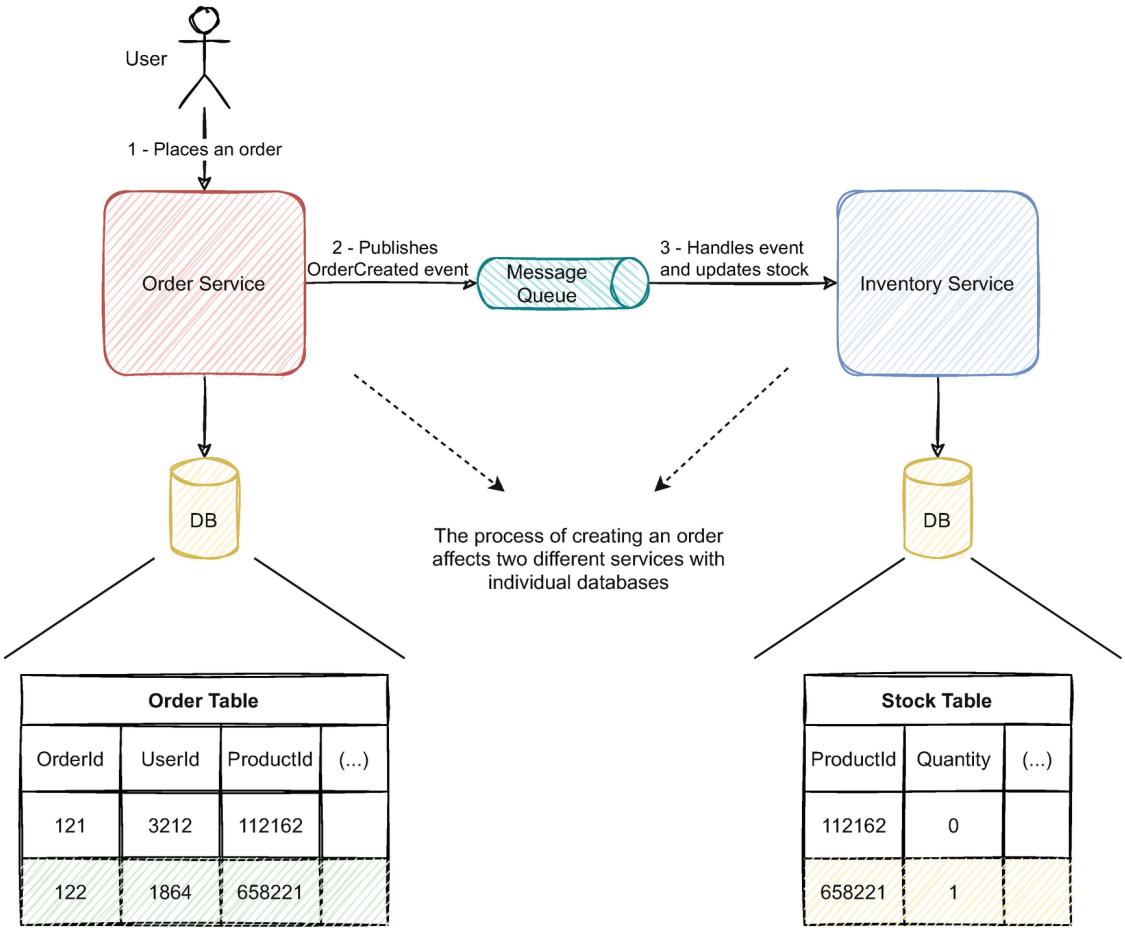

Figure 4-2 To accomplish the order process, both services need to reflect the changes in their databases

Since we separated the order and inventory domain into two distinct services, the order and inventory data aren't in the same database anymore. The segregation creates the challenge of how we can guarantee the strong consistency of fulfilling an order. The example only illustrates two services, but there might be processes that need the interaction of several different domains; for instance, if we needed to apply discounts, change the user information, etc., it might affect several distinct services. When

creating an order with a single database, we could simply wrap a transaction around every table that needed changes, and we would be sure all changes would happen atomically. However, with a distributed architecture, we can't have the same guarantees. The alternatives to manage these kinds of issues come at the cost of added complexity as we will discuss in the following sections.

4.1.1 Why Move from a Monolithic Database in the First Place?

Besides the monolith's limitations, we discussed in Chapter 1, why forfeit the consistency guarantees provided by ACID anyway? In use cases that need strong consistency, we might just be better off by keeping that data together, even in a distributed microservice architecture. The strong consistency ACID provides can only be achieved by having data in the same place. Don't be dogmatic in adopting a distributed solution; if there is a critical use case for strong consistency, consider if it is feasible to maintain the data together and benefit from the properties associated with it. However, this is the exception; most use cases can live with weaker consistency guarantees distributed systems require; most of the time, they are good enough as long as the system is performant (we will further discuss this in Chapter 5).

In use cases that deal with vast amounts of data, we might need to forfeit the ACID properties anyway. The purity of the third normal form we discussed at the beginning of the chapter often doesn't have a place in the real world. Small and medium-sized data undoubtedly benefit from it, and in those situations, we often benefit from designing the database schema under those principles. But when faced with huge amounts of data, we quickly face its limitations. Often queries on database schemas designed under the third normal form need several joins between the different tables. I'm sure we all relate to debugging a given non-performant query with an obscene amount of joins in the past. A high number of joins between tables with millions of records requires excessive effort from the hardware. With increased usage, we quickly find we need better hardware. The only way to typically scale a relational ACID database is often vertically, which becomes very expensive very fast.

What were the traditional approaches to deal with this issue? We often add sharding and replicas to our databases – replicas that suffer from the replication lag the same way as event-driven architectures do but in systems that were not designed to do so. Often we need to rewrite or do dubious workarounds to afford the replication lag. To tackle slow queries, we denormalize data and optimize how data is persisted to enable high read performance. As we throw our referential integrity and lack of duplication out of the window, often against our inner values, which are chiseled in the properties of the third normal form, we say, "well just this time, this is the real world, sometimes we must forfeit these kinds of theoretical concepts."

I saw terrible implementations in the past due to a stubborn dogmatic approach to theoretical concepts, and I'm guessing at one time or another you saw it too. Sometimes in the real world, we need a pragmatic approach to the solutions we implement. But these kinds of solutions of "implementing as we go" often fail due to the lack of a deliberate long-term strategy. Instead of trying to fit performance and scalability into solutions that were not designed to fit them, and only when they become an issue, we should approach it as a property of the solution we create.

Deconstructing a monolithic database is a step in that direction. In event-driven architectures, besides having synergy with dividing an extensive database into smaller ones, the focus of the data is in the event stream, enabling the possibility to disseminate data without relying solely on the database infrastructure. We do gain complexity, and we might lose the stronger consistency properties ACID provides, but we won't constrict the business growth to a halt. If you don't see and don't ever foresee the database being a limitation, then you are probably better off not dividing it at all.

4.1.2 The Limitations of Distributed Transactions

When faced with the challenge of guaranteeing atomicity between two different databases, some people might be tempted to use distributed transactions. They provide a way to reason with a distributed system featuring several databases the same way we would with a single ACID data-

base. Distributed transactions are often implemented with the X/Open XA standard,[2] which uses a two-phase commit to ensure the atomicity of all the transaction's changes. However, the two-phase commit protocol doesn't provide all ACID guarantees. It also has numerous failure modes that might need manual compensation and leave the data in an unexpected state. This subsection will detail how the two-phase commit protocol works and its limitations on a distributed system.

The two-phase commit protocol typically uses two types of components: the transaction manager and the participants. The transaction managers coordinate with the participants the protocol's implementation for each transaction. Unsurprisingly, the protocol has two phases: the voting phase and the completion phase. In the voting phase, the transaction manager communicates with the participants to confirm that a given change can be made. Let's use the same example we discussed earlier in this section to illustrate this. Figure 4-3 depicts how this would play out with the stock and order tables.

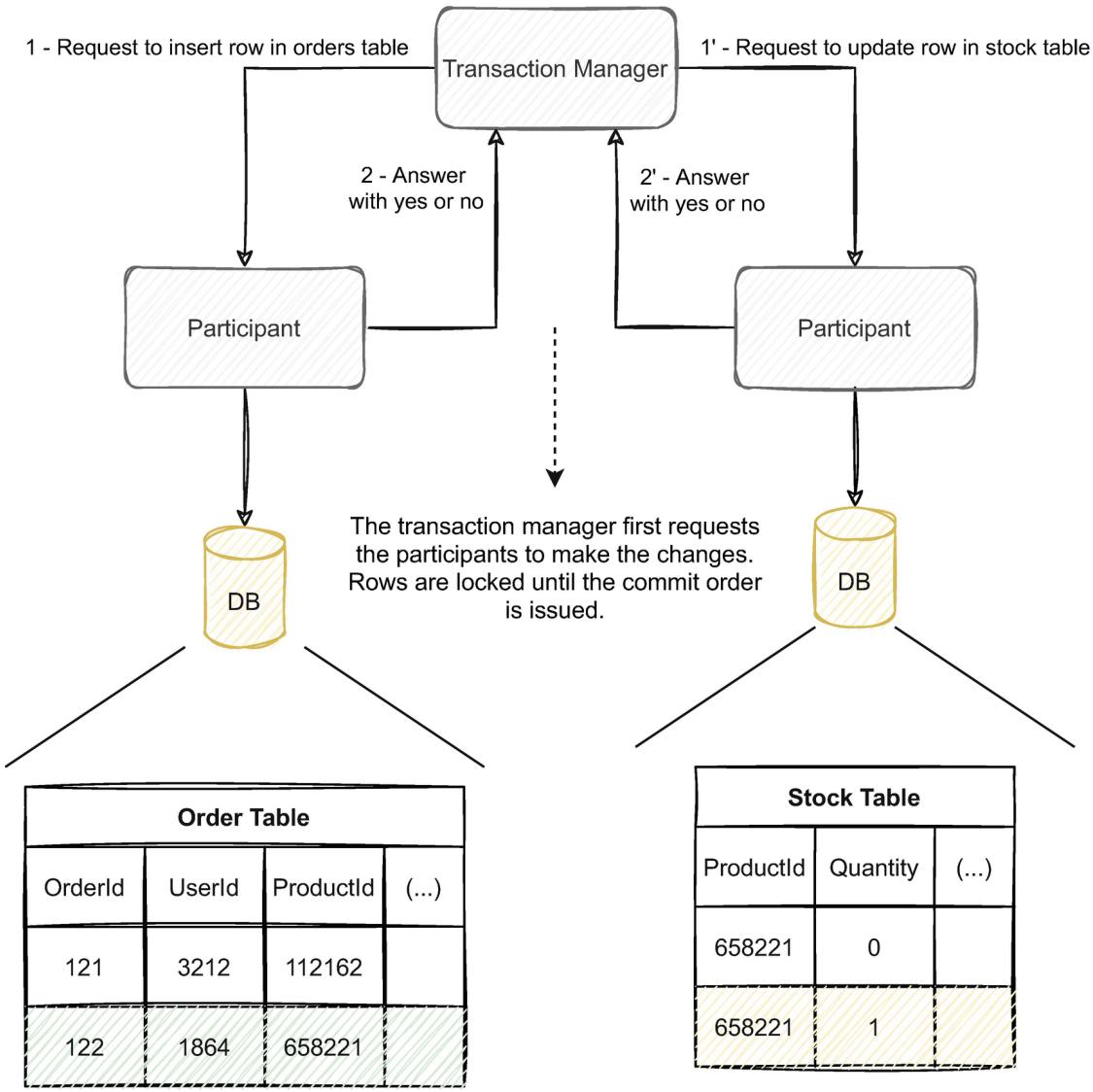

Figure 4-3 Voting phase of a distributed transaction between the order and stock table

The transaction manager requests the order and inventory database participants to insert the new order row and update the existing stock row, respectively. If one of them responds negatively, the transaction aborts. If both participants respond positively, then the transaction manager will advance to the completion phase.

In the completion phase, the transaction manager will request both participants to commit their changes, as illustrated in Figure 4-4. If a participant is unable to commit their changes for some reason, the transaction manager will have to request a rollback from all participants. A rollback will undo any changes that were made or release any locked resources. The transaction ends with all participants acknowledging the transaction manager's changes by successfully committing or roll backing the changes.

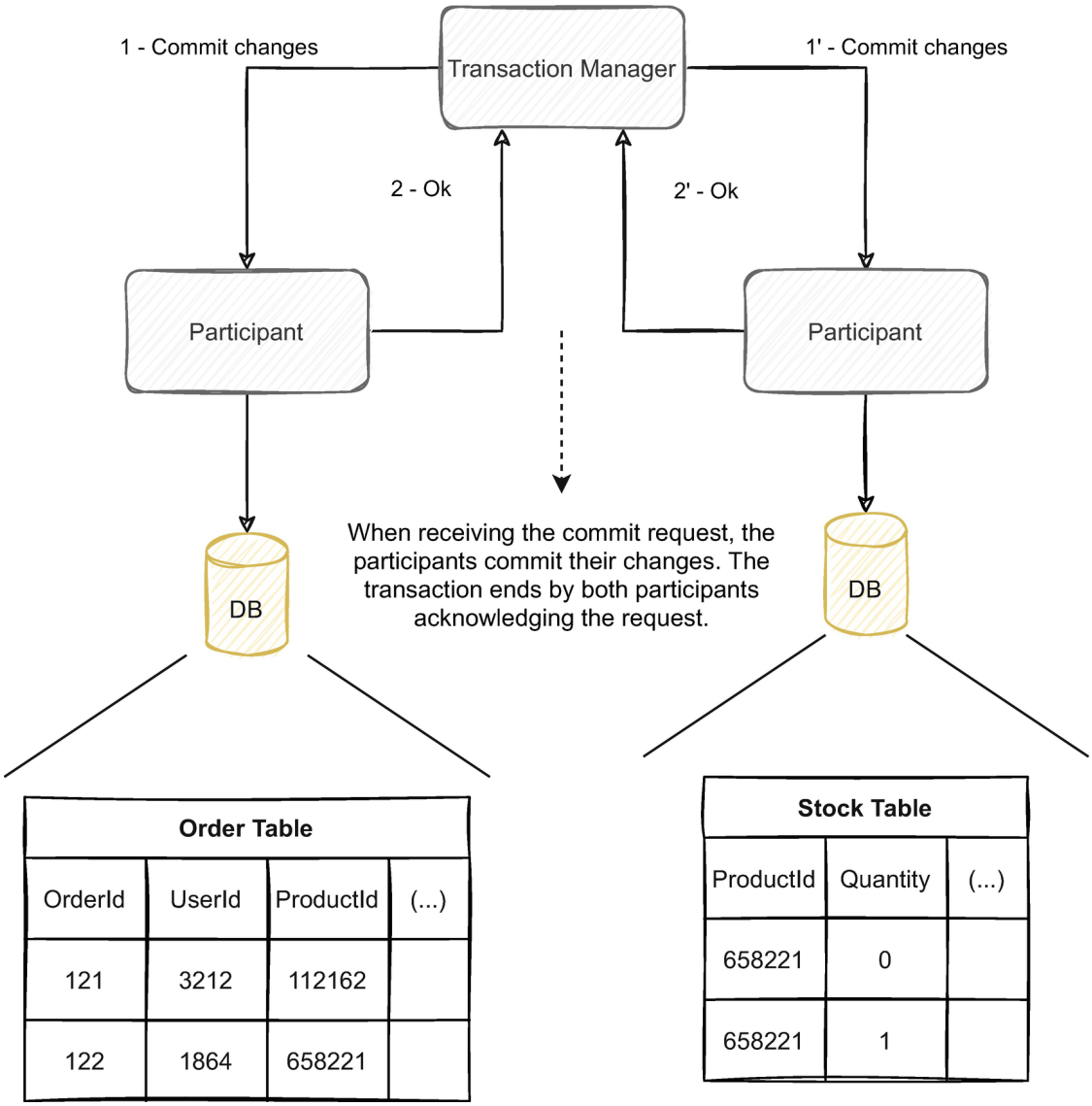

Figure 4-4 Completion phase of the same distributed transaction between the order and stock table

This protocol raises several complex issues and might leave the data in an unexpected state. Since two different processes handle both commit

requests, they can happen at different times. So we don't have the ACID properties we discussed earlier anymore. A client can request the order information without the stock being updated, for example.

Also, since both operations occur separately, an arbitrary amount of time can pass between the voting and completion phases. Concurrent requests may try to change the same data while the transaction is running. For example, if someone submitted an order for the same product simultaneously, another process might try to change the stock line while the transaction is still running. To avoid concurrent changes, the participants often lock the resources until the transaction ends. That means while the transaction is occurring, every other request to access the same data will have to wait. We all know the associated performance problems with traditional transactions, but taking them to a distributed environment where they are susceptible to the network conditions and delays might severely impact the system. Two-phase commit protocol is suited for fast operations; the longer they take, the longer the data is locked. The examples we discussed are with two participants, but the more participants involved, the longer the transaction takes.

Being susceptible to the network conditions also opens a wide range of new failure modes. Jepsen, a set of tests and analysis on distributed consistency on many popular databases, did an analysis[3] on Postgres which uses a special case of two-phase commit and highlighted some of these problems (there is also another interesting analysis on YugaByte DB[4] which uses a homegrown commit protocol based on two-phase commit highlighting similar problems). For example, in Figure 4-4, the participants have to acknowledge the commit to the transaction manager. If an acknowledgment is lost, for example, the participant suffered a network partition (more on network partitions in Chapter 5), the transaction fails. But the participant might already have committed the data and has no way to receive the rollback request, which can lead to unexpected results and produce an invalid state that must be solved manually.

Another issue with the two-phase commit protocol is the transaction manager. When several different services are involved in multiple transactions, the transaction manager acts as the coordinator between these

different services. Event-driven architectures are naturally decoupled and enable us to build independent services; having a single component in charge of coordinating synchronous operations between the services is a step back in this mindset. Distributed transactions often raise more issues than the ones they solve. As we discussed in Subsection 4.1.1, if we really need the ACID properties for a given use case, we are often better off by maintaining that data together in a database that affords ACID guarantees. But don't be tempted to see this is always the case; most often than not, we can live with weaker consistency guarantees, as we will discuss in Chapter 5.

4.1.3 Managing Multi-step Processes with Sagas

We discussed the limitations and why we shouldn't use distributed transactions, but what is the alternative? Business processes often span several different services. How can we maintain the consistency of those processes when multiple services are involved? This subsection will discuss how Sagas can be an alternative approach to distributed transactions and coordinating synchronous locks.

Sagas were first introduced in a paper[5] by Hector Garcaa-Molrna and Kenneth Salem, where they suggested the use of Sagas to manage long-living transactions. The paper argues the use of smaller short-lived transactions as an alternative to long-lived transactions. Instead of having a single transaction that lasts a considerable amount of time, we can have smaller ones for each step of the more extensive operation, minimizing the amount of data affected by locking.

Sagas are a sequence of individual operations to manage a long-running business process. In a distributed architecture, each of the Sagas' operations is carried out by a different service. In an event-driven architecture, the services typically are orchestrated or choreographed through events to achieve the larger business process. The business logic itself is located inside the services; each service will manage and validate its own domain. We define a compensating action for each step of the business process. Suppose a given service fails to perform the action due to techni-

cal or business reasons. In that case, the Saga's currently completed steps will execute the compensating action to leave the system in a stable state.

When used with DDD, Sagas manage operations across several aggregates between different bounded contexts. When an aggregate is affected by an event of a different domain, and we need to translate it to a command, it is often seen as a Saga. Operations that affected multiple aggregates are also commonly seen as a Saga. Although Sagas can be applied to reflect changes to multiple aggregates, the most common use case is to manage a long-lived business process between several bounded contexts.

Each operation of the Saga occurs separately and independently. This means the entire Saga isn't an ACID transaction nor enjoys its guarantees. As the Saga completes its steps, the changes each step applies are immediately available, even before the entire process finishes. Each step can have strong consistency guarantees and be inside the context of a transaction if the database in question supports ACID guarantees.

Sagas aren't strongly consistent as a whole, but they provide a way to understand and model a business process in a way we can act when something goes wrong. Each of the Sagas' steps can have a compensating action we can trigger if one of the following steps fails. This way, when a given step fails, we can maintain the consistency of the whole process. Let's illustrate with an example. Figure 4-5 depicts an example of an order submission process.

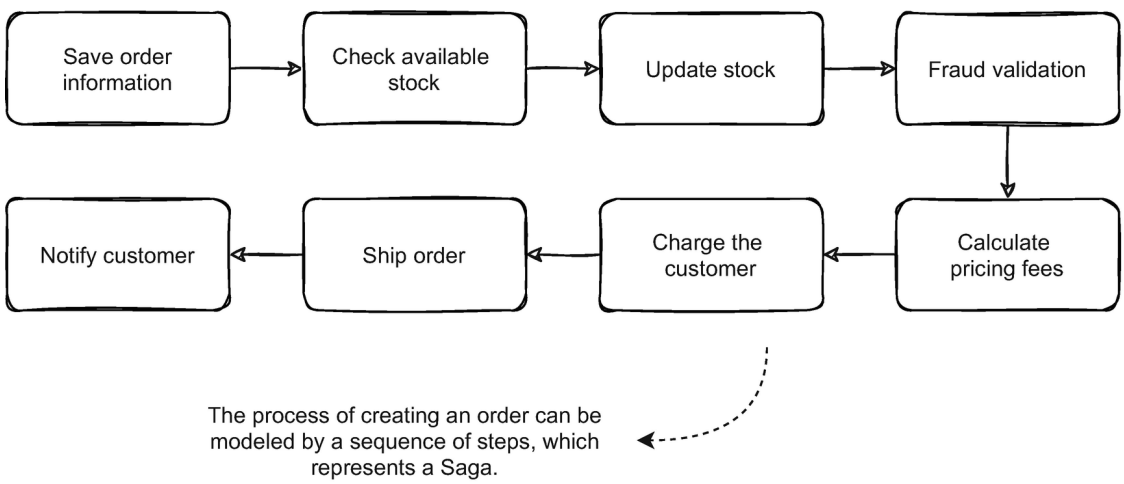

Figure 4-5 An example of an order processing Saga

Each step in the Saga might map to an action in a specific service or in different boundaries. For example, the order service might be responsible

for saving the order information, but checking the available stock and updating it might be the responsibility of the inventory service.

In a monolithic approach, the example in Figure 4-5 could be implemented using a single transaction. If a step fails, the whole transaction would roll back and would maintain the consistency. Using a Saga, we must implement a compensating action for every relevant step. The rollback process has a workflow of its own depending on the stage that fails.

Figure 4-6 illustrates an example of a rollback in this workflow. Let's say the order the system was processing failed in the fraud validation step. The previous steps were already completed; for example, the stock has already changed.

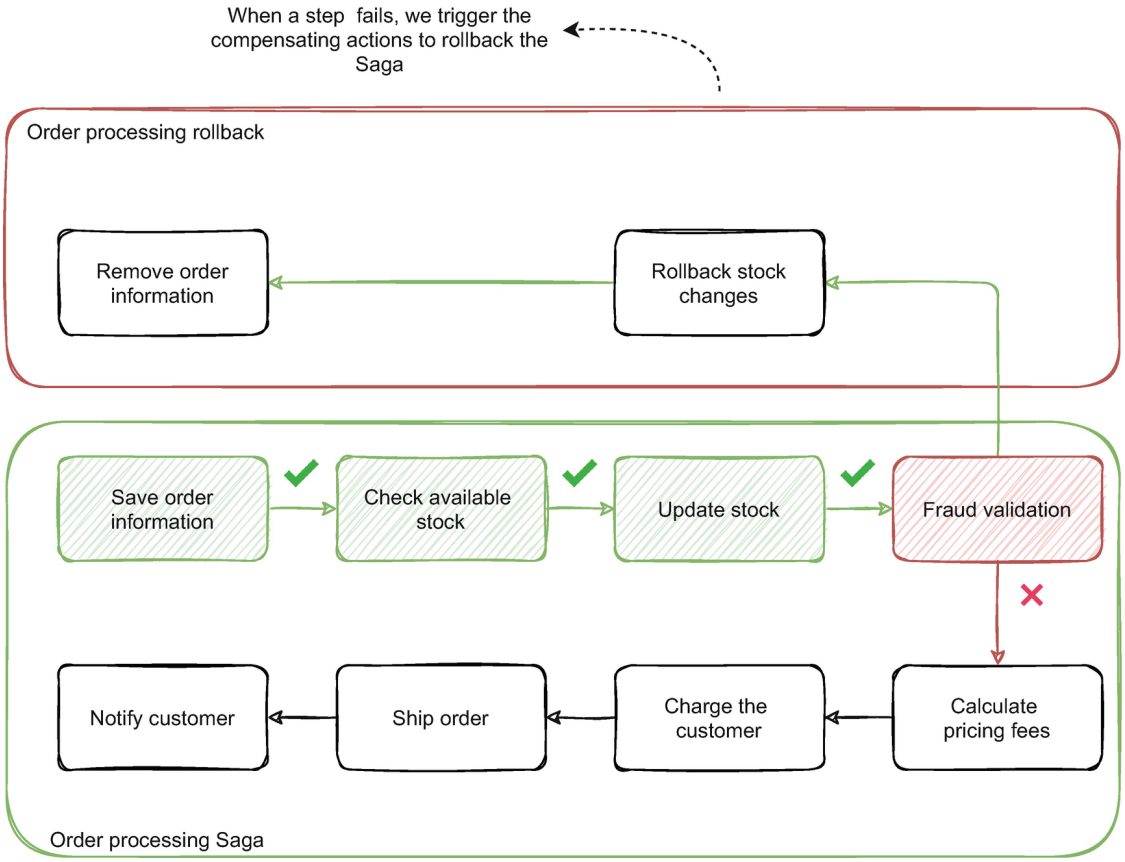

Figure 4-6 Rollback of the order processing Saga by failing to do the fraud validation

How can we roll back the process? To maintain the system's consistency, we would trigger the rollback process which would do a compensating action for every step that needs to. In this case, the fraud validation step's failure would trigger two compensating actions to undo the stock changes that happened on the third step and remove the order information that happened on the first step.

Stateless and idempotent actions, like the step to validate the available stock and calculate pricing fees, are more straightforward to handle since no state rollback is needed. The order we design the Saga's steps is also important. Depending on how we model the workflow, the rollback process might be more or less complicated. For example, if fraud validation was the first step, we wouldn't need to do any compensating action, and a rollback process wouldn't be needed. When modeling a Saga, a vital consideration is to think about the process and how we can minimize the chance of rollback and the number of compensating actions we need to do.

An important consideration is what kind of reason can make a step fail. If it fails due to domain validations or business rules, the compensating actions will suffice to put the system back in a reliable state. However, if steps can fail due to technical reasons, for example, if a given service is offline or unreachable, one can ask what we should do when the rollback process fails. For example, in Figure 4-6 after the fraud validation fails, what if the inventory service is unreachable or due to a network issue, or a failure reaching its database, or unable to perform the action due to a bug? We couldn't undo the stock changes done by the initial steps of the Saga.

Working with event-driven services and the natural decoupling and asynchronous nature they imbue in the systems provides a way to be more resilient to transient failures and enable us to build more failproof processes. Retrying the operation is easier, and even if the retries fail, the event is persisted in the event stream. If we need to reprocess a given action, we can retry the same event as long as the event handling is idempotent. In a synchronous process, the Saga might hang until a retry would work or might need ad hoc processes to retry the Saga. The asynchronous nature of the event streams naturally provides a way for the system to continue to respond without blocking while processing the workflow or cascading the failure to the other components. We will further discuss resilient message processing in Chapter 7.

4.2 Event-Driven Orchestration Pattern

In Section 4.1, we discussed how we could use Sagas to model a business process that needs the intervention of multiple independent services. Sagas usually come in two variants: orchestrated or choreographed. In this section, we will discuss how we can use orchestration to implement a business process.

Orchestration uses a primary component to manage the steps of the process. This component instructs the other services to start new steps in the process and triggers the start of compensation actions when needed. It works much like a supervisor that oversees the whole process and delegates the actions to subordinate services.

Let's use the same example about order processing we discussed in Section 4.1. Figure 4-7 illustrates how we could model that process with the orchestration pattern.

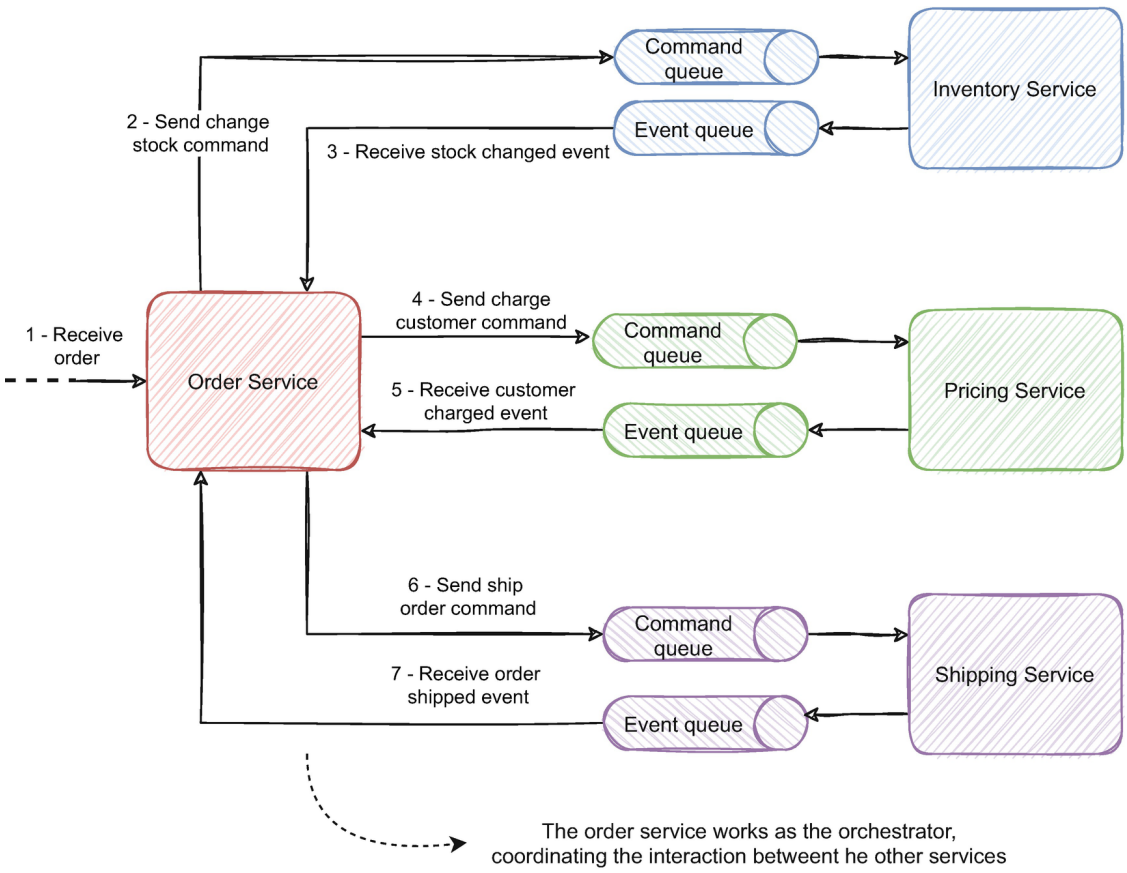

Figure 4-7 The order process workflow implemented using an orchestration pattern

In this example, the order service assumes the orchestrator role and coordinates changes between three other services: the inventory, pricing, and shipping service. In Section 4.1, we modeled the order processing Saga depicted in Figure 4-5. The first step of the workflow was receiving

the order submission request. The order service would handle the request and coordinate with the other services to accomplish the process steps.

Instead of sending commands, this pattern could be implemented by sending an order created event to each service. But events aren't sent, they are published, and services react to them. If we want to change another service or domain, we should reflect that with a command rather than an event (as we detailed in Chapter 3). If we used an event, we would likely fall into the passive-aggressive events anti-pattern.

Notice that we also merged several steps into one operation (this wasn't just because it was simpler to draw the diagram, despite being a concern). Although we might conceptually model the order process with every step, we might want to merge some of them when implementing it. For example, if the check and update stock were two different operations, two orders for the same product at the same time might concurrently pass the validation. Since both operations belong to the inventory bounded context, we can implement them with one command.

The stock operations relate to each other, but what if we had other operations with the same challenge that we couldn't merge? If the order service had to validate the stock and then request it to change, it could lead to orders for products without stock. We could trigger an error and roll back the process by triggering the compensating actions, as discussed in Section 4.1, but that's a design solution for a technical problem. It is an option, but compensating actions are often better for undoing domain issues; for example, the customer doesn't have enough money to make the purchase. We explore eventual consistency and concurrency issues in Chapters 5 and 6.

The interaction between the services is fully asynchronous, and the process advances with the asynchronous handling of events from each participant service. The order service requests the stock changes; once they are processed, the inventory service publishes an event signaling those changes, which the order service uses to advance to the next step, requesting the pricing service to charge the customer in the same way.

The same interaction occurs with other services until the order is fully processed.

A strong advantage of this pattern is the ability to easily monitor the process's status since only one service is responsible to manage it. It would be straightforward to understand each order's current step and result by looking only at the order service. It is also easy to reason with the logic of the order business process; we can quickly have a high-level vision of the process by only analyzing the order service.

The trap in this pattern is building the orchestrator to be an aggregator of all the process's logic. Code often has gravity; it pulls more code toward it. The more you add, the more it pulls. Stay with it and you likely end up with a monolith. These types of orchestrator services often become a quick solution to include new features, sometimes features that belong somewhere else.

Each service is responsible for maintaining its business rules and its domain's consistency. The decoupling provided by the queues promotes that, but it is important to include new developments in the appropriate domain. The order service only requests that changes be reflected in that domain and doesn't explicitly say what that domain should do. For example, it instructs the pricing service to charge the customer, but only the pricing domain knows the country's taxes, discounts applicable to the customer, and valid payment methods. Having a central piece to manage the workflow provides a tempting component to add logic that otherwise belongs to other domains.

A way to manage this is to use an approach similar to the one Cassandra uses to manage writes. Cassandra is a popular NoSQL database that uses a coordinator node concept instead of having the typical primary-secondary topology like SQL Server or MongoDB. Primary-secondary topologies require that all writes go through the primary. In Cassandra, any node can handle writes by coordinating the write to the node that holds the data.

We can use the same concept by having different services managing different business processes. It is understandable to include the order ful-

fillment process in the order service since the order's workflow is often closely related to the order domain. By following the same reasoning, we could include the update of the pricing fees business process in the pricing service, for example. Having a central agnostic orchestrator that handles every workflow or a large number of them is an anti-pattern and something we need to avoid. Falling in that anti-pattern will rapidly take us to a distributed monolith where we despair with both the disadvantages of monoliths and distributed systems.

If any of the services failed to process, the service would signal that failure sending an event. In that case, the order service would handle that event and trigger any corresponding compensating actions.

A synchronous process would wait for each service's response to complete the process. Even if we implemented a job style processing (having a job that checks the process's status from time to time), the orchestrator would have to poll the services to know when they finished the operation. The fully asynchronous nature of this process makes it easier to manage the load by distributing it to the dependent services, making it easier to scale the parts of the system that need scaling. The queues would also absorb the added scale; a load peak would only be reflected in the system by an increase in lag. The flow is also more organic since there is no pooling involved and the orchestrator manages the progress by reacting to the events being published by the services.

This pattern is an interesting solution to complex processes that need a central supervisor; however, we need to be aware to not fall into the pitfall of feeding the orchestrator beyond measure. Small and trivial business processes will hardly benefit from the orchestration. Often, not having central pieces that orchestrate logic helps to encourage the distribution of domain logic and the organic evolution of an event-driven architecture.

4.3 Event-Driven Choreography Pattern

In the last section, we discussed how we could use orchestration to model a complex business process. In this section, we will discuss choreography

as an alternative to orchestration. We will detail how the example of the order processing we discussed before looks like using choreography.

When using choreography, the services involved react to each other to complete the sequence of steps. Rather than a central piece telling them what to do, like orchestration, the business process is autonomously coordinated; each service knows what to do once a service finishes its action. Instead of a supervisor delegating tasks to subordinate services, the services themselves know when to perform their tasks by reacting to the preceding service's events. This pattern's foundation is closer to the event-driven mindset than the previous one due to the organic flow of events without a commanding central piece.

Figure 4-8 illustrates the same order fulfillment process using a choreographed approach. There is no central component that manages the workflow of the events and oversees the process's progress. Instead, each service reacts to the changes happening in the ecosystem to accomplish its purpose.

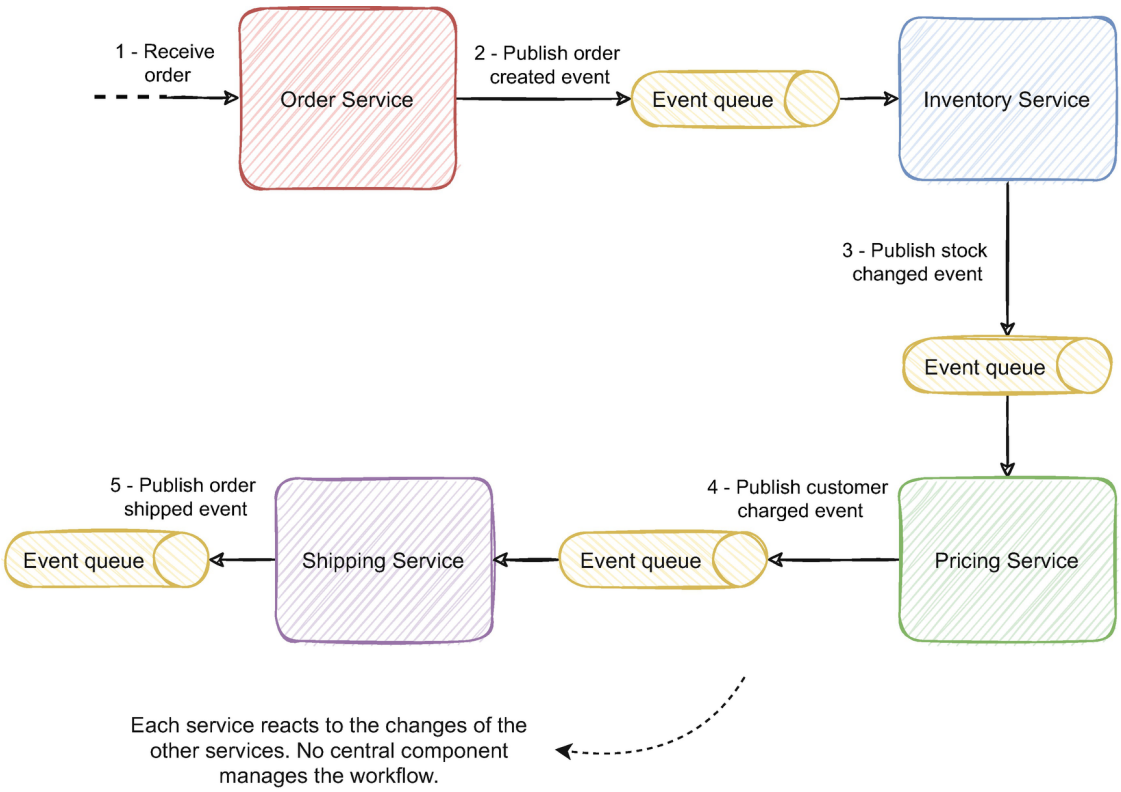

Figure 4-8 The order process workflow implemented using a choreography pattern

The order service after saving the information publishes an event announcing the order was created. The inventory service reacts to this event and changes the stock. Once the stock is altered, the pricing service

reacts to the stock changed event to calculate the pricing fees and charge the customer. Finally, once the customer is charged, the pricing service publishes an event that the shipping service will use to ship the order and complete the process.

If something goes wrong and we need to roll back the process, an event from the service that failed to process would trigger the other services' compensating actions. For example, if the pricing service determines it is a fraudulent order, it would publish an event signaling the process's failure and the inventory and order service would react to it, triggering their own compensating actions.

This approach solves the issues we mentioned about logic being centralized in one place. Since there is no central orchestrator, the logic is naturally distributed throughout the services. Using orchestration, as we implement new functionality, one way or another, the team managing the orchestrator service ends up influencing the behavior of the other domains, often heavily biased on their own. Choreography often facilitates the evolution of the domains based on what is important for each one, without a central process manager's influence.

One of the caveats with this approach is we no longer have a straightforward way to understand the workflow. With orchestration, by analyzing only the orchestrator, we can follow every step of the workflow. With choreography, the workflow is embedded in the behavior of each service. The decoupling provided by the event queues also makes this process difficult. This is one main concern with event-driven architectures; the highly decoupled nature hinders the understanding of the event stream's high-level flows. We will detail strategies to tackle this later in this chapter.

The compensating actions might be harder to implement since each service will have to implement or react to each new step or change in the workflow. For example, if we added a new final step to manage the order returns in a new service, the other services would also need to react to that new domain's failures, if applicable. On the other hand, it promotes the mindset to reason with each domain independently; on an order return, how should the inventory domain react? It promotes the discussion

on how that change affects each domain instead of delegating that responsibility to the orchestrator.

Another main concern with choreography is the ability to understand the current state of each Saga. With orchestration, if we needed to know in which step the order process was in, we would likely implement that feature in the orchestrator. Since it monitors every step and the overall workflow, it would be straightforward to know each order process step. With choreography, there is no prominent place to obtain each state. One solution is to listen to every event being published by each service and materialize them in an aggregated view, as illustrated in Figure 4-9.

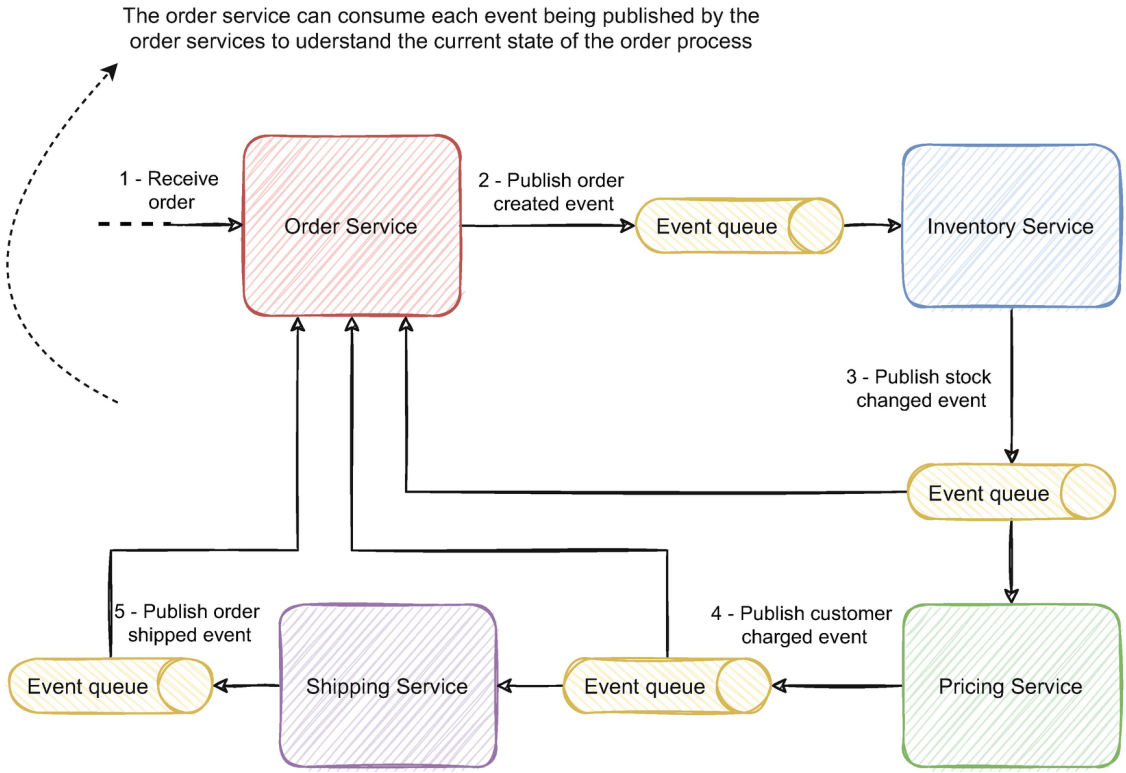

Figure 4-9 The order service can listen to the other events to understand the current state of the order process

The example shows the order service listening to every event but could be a different separate service to consume the events and materialize them into a view. The component handling the events and materializing the state will likely have to understand the higher-level workflow but doesn't manage it as an orchestrator does. Using this solution is still choreography but definitely has a halfway feel between the two approaches.

Simpler processes don't suffer as much with this solution's caveats. As the workflow grows more complex, the more challenging it is to deal with

the drawbacks. Overall orchestration tends to be a common pattern in event-driven architectures, and it synergizes well with its mindset. The caveats can be tackled by the approaches we discuss further in this book. However, if we have a requirement that needs a central component to manage the process and keep its state, an orchestration approach might fit better.

4.4 Event-Driven Microservices: Orchestration, Choreography, or Both?

In Sections 4.2 and 4.3, we detailed how orchestration and choreography work and how we could apply them. But should we stick to just one approach? This section will discuss how typically they are implemented in an event-driven architecture and how they can coexist together.

In event-driven architectures, we typically use choreography since it naturally fits the event-driven mindset. We publish events to event streams and applications plug into those streams to access the data. One main benefit in adopting an event-driven architecture is the highly evolving nature and decoupled properties they provide. Typically, business processes have no central orchestrator. The lack of a central piece avoids the logic of being centralized in a single place and doesn't imperil those benefits.

When we continuously add functionality and new services to the architecture, an orchestrator typically grows more complex. As we add more people and more teams, we slowly start to struggle with the monolith's challenges in the orchestrator. Not having a central piece eases these issues and enables the domains to grow naturally. It gives the team's autonomy to decide what's best for their domain and deliver features autonomously without having to deal with a central shared piece.

An orchestrator outside the boundaries, coordinating the changes between them, will likely grow and potentially turn the boundary service's domain anemic. Having a different component in each boundary as the orchestrator for different business processes can help with this. Still, it also means different sources make changes to the entities, which can be

hard to manage. Often orchestrators fall into the same difficulties traditional SOA with ESB usually fall. As you might recall, we discussed SOA with ESB in Chapter 1. Typically, ESBs are a place to centralize business logic and tend to grow as we add more and more features. Orchestrators are a great place to centralize that logic and have anemic satellite services.

Depending on the use case, choreography often tends to be the standard in event-driven architectures. As boundaries grow, it would likely be problematic to manage a central piece. However, it doesn't mean orchestration doesn't have a place in it. It is often useful to apply it in a more localized context, for example, inside a boundary. Figure 4-10 illustrates an example of this situation.

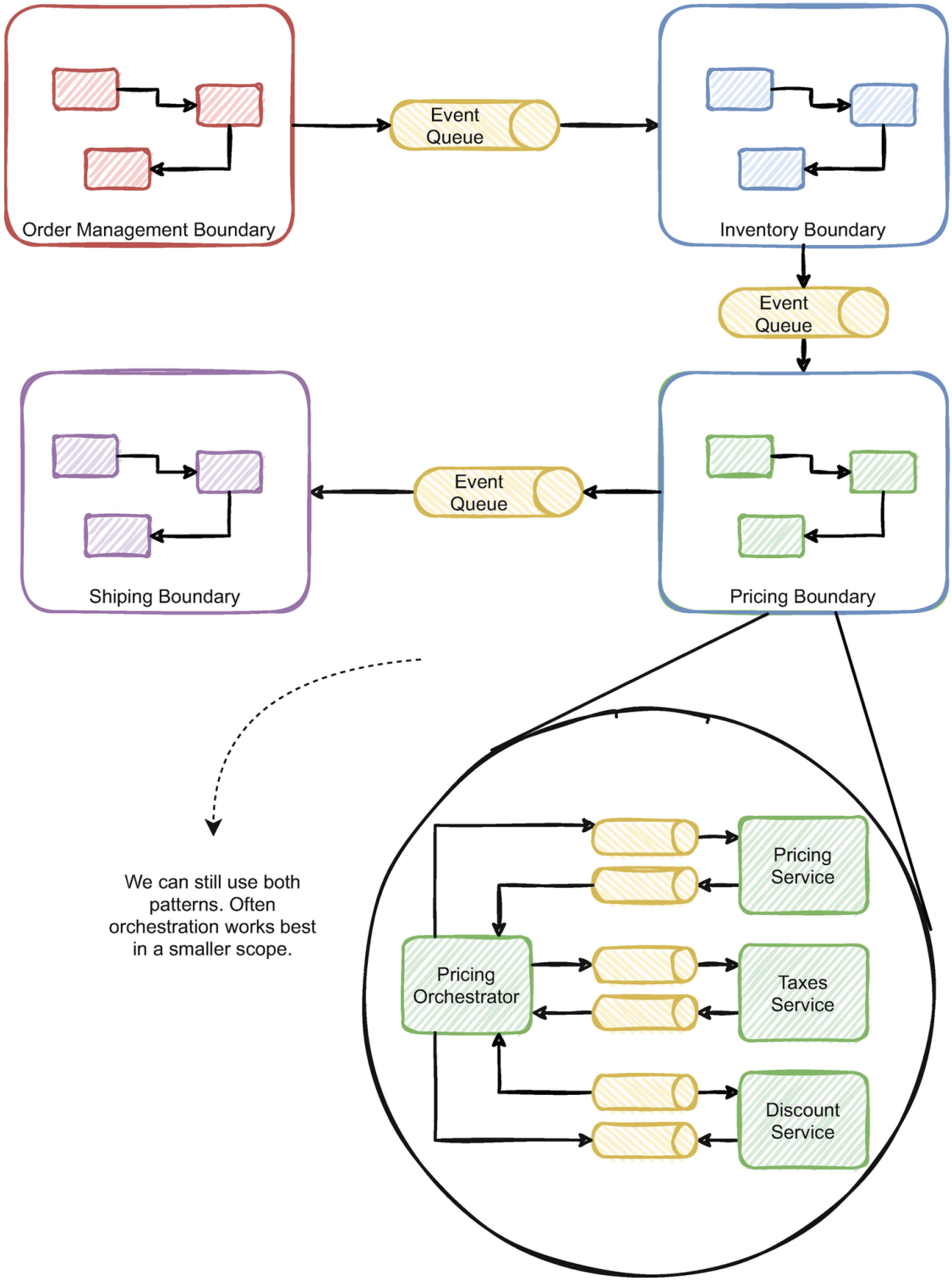

Figure 4-10 Example of applying both patterns. We applied orchestration in a smaller scope

Figure 4-10 illustrates the same example we discussed before, but instead of a single service, each bounded context is composed of several services. We use choreography between each boundary, but inside a boundary, we can apply orchestration, as illustrated in the pricing boundary. The more extensive higher-level process uses choreography, but to manage a smaller scope and more localized process, we could use orchestration like the pricing calculation.

This use case is just an example, but the main line of reasoning is to use choreography as default and in higher-level processes and apply orchestration on a limited scope. This approach allows a high decoupling between the services and enables the team's autonomy at the higher level where multiple teams are involved. And it uses orchestration on a smaller scope, where a small number of people are involved and have the autonomy to release features without impacting several areas.

4.5 Data Retrieval in Event-Driven Architectures and Associated Patterns

Until now, we described how event-driven architectures work, and we briefly mentioned queries as a common artifact of these architectures in Chapter 3. However, we didn't detail how to provide query functionality driven by events. Queries are a common requirement in most applications to provide data to the user or other services. CRUD-based services simply query the database and return the requested data synchronously. Event-driven architectures add a layer of complexity to the traditional approach since data is distributed throughout several services and likely eventually consistent. In this section, we will describe common patterns that synergize well with the event-driven approach.

Chapter 2 discussed how to move from a monolithic architecture to a distributed event-driven microservice architecture. We discussed how to split the monolith into several services and how to deconstruct the database; the data associated with each domain was moved to the respective service. When in a monolith and a monolithic database, all the data is centralized in the same place, and it is straightforward, from an implementation perspective, to obtain and join data. Typical monolithic platforms simply expose information through a synchronous API like REST or SOAP. This approach is arguably the most common and straightforward approach. It suffers, however, from all the scaling limitations we discussed in Chapter 1.

By deconstructing the monolith and its database, we are now faced with a challenge; how can we get an aggregated view of the data? We have a set of independent services, each owning its domain's data; each service will only be able to

return information about its domain. For example, Figure 4-11 illustrates four different services; the UI needs to list a page with all the available products with their current stock and price.

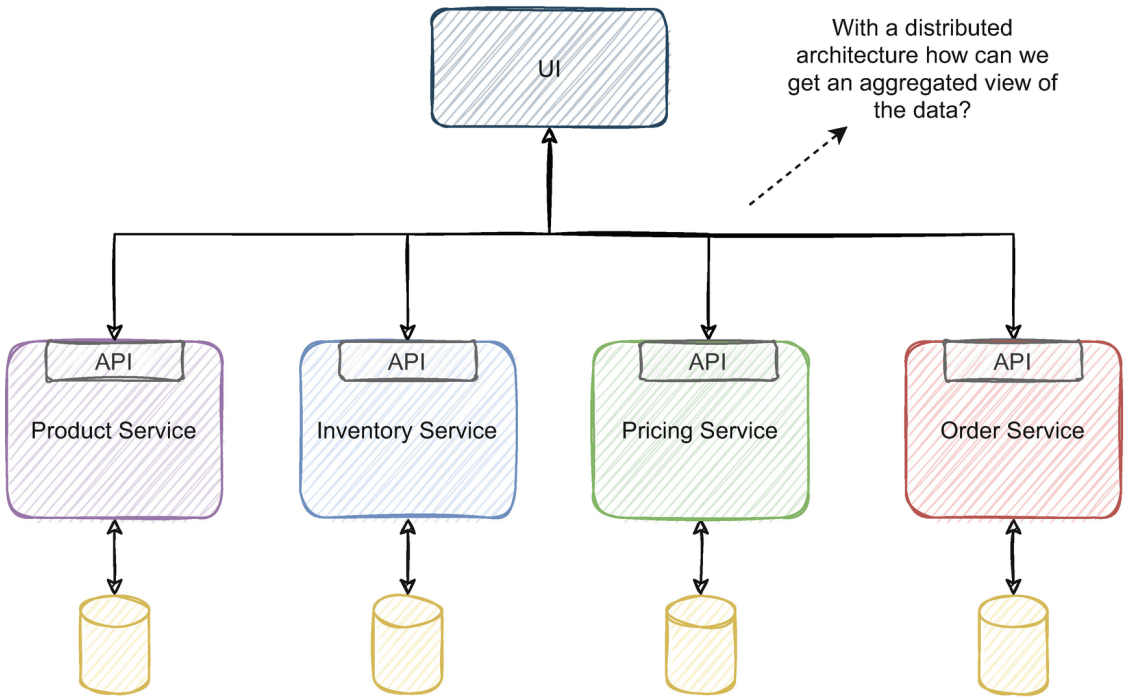

Figure 4-11 Having a distributed architecture poses challenges on how to have a denormalized view of the data

A possible approach to solve this challenge is to use the same strategy as illustrated in Figure 4-11. The UI application can fetch information from every service and merge and display it in a UI. This approach is usually called the API composition pattern, and it's probably the solution that first comes to our mind.

This kind of approach has severe limitations; as you can imagine when we deal with large amounts of data, this approach rapidly struggles. If we need to display a listing page with all the products, let's say with millions of products (although a lot less would likely suffice to make the application struggle), the UI would likely need to load several thousands of those items to memory. Imagine if the user needs to filter all products with stock and order them from the lowest to the highest price, a standard functionality in most eCommerce websites. The UI application to support such queries would likely have to fetch thousands of records from each service and filter them in memory to display accurate records with filtering, sorting, and paging. If several users did this simultaneous-

ly, the application would likely struggle with the performance overhead and the memory constraints.

API composition can be relevant for small data sets and can be a cheap solution for some use cases (although one can ask if we are using small data sets, why are we using a distributed architecture anyway?). Even for one product, it would mean three requests to different services. The additional requests can have a performance overhead in the UI and likely lead to poor user experience. Data at scale and complex searches require a different approach.

At the beginning of the chapter, we discussed how we needed to sacrifice the third normal form's values in the altar of performance and how we created denormalized models for specific querying requirements. Event-driven architectures with persisted event streaming provide a sustainable way to implement this. Since every event is readily available even after consumption, we can use those events to enrich a service's information.

Let's first look at a typical implementation of a synchronous query implementation. Figure 4-12 illustrates a service, the product service, exposing an API for consumers to use. Other applications can use that API to change the service's data or to retrieve information from the service.

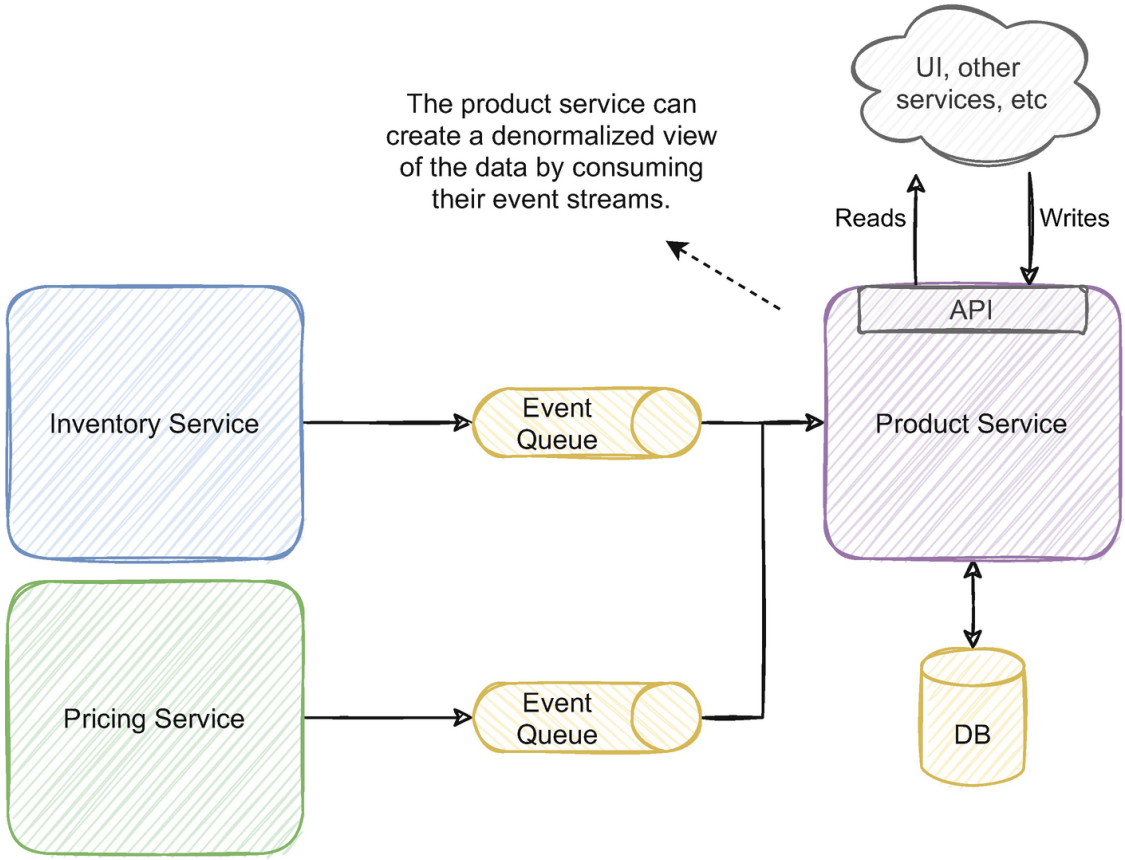

Figure 4-12 The product service can build a denormalized view of the data by handling events from the other services

In a distributed event-driven architecture, it isn't as straightforward since different services own different data. The example in Figure 4-12 handles events from an event queue to enrich its read model with additional data the service doesn't hold. For example, the product service might own all the data related to products, but the inventory information might be managed in a different service. Like a product listing page on a UI, queries to the product service might need the stock information for each product. The product service can enrich its information with stock information handling the events being published by the inventory service, thus the event queue.

Resuming the example we discussed in Figure 4-11, the product service can create a denormalized view of the data with all the information needed for the UI requirements by handling both inventory and pricing events. Filtering and sorting would actually be viable since there is a persistent denormalized model.

4.5.1 CQS, CQRS, and When to Use Them

An interesting pattern that usually has high synergy with this kind of approach, and is usually mentioned along with DDD, is CQRS (command query responsibility segregation). CQRS originated from CQS (command query separation), and people often are generally confused when distinguishing the two patterns; once you read this section, let's hope you're not one of them. This subsection will detail both patterns, what we can benefit from them, and how to apply them in the same product listing example we discussed before.

Bertrand Meyer formulated CQS in his book[6] *Object-Oriented Software Construction*. This principle states that methods can either be queries or commands but never both. Queries return a value but don't change state, and commands change state but don't return any value. Basically, asking doesn't hurt, and if you're not asking, don't tell.

CQRS originated from CQS and introduced a slight change; CQRS entities are split into two main concepts, commands and queries. Having a clear separation between the objects in code is handy because we can use queries at will; we are sure they don't change state. While with commands, we can be more careful about where to use them. It is also easier to find the code paths that change state and the ones that retrieve state. CQS can be a valuable approach to apply locally in specific services, the overall organization helps to reason better with the service, especially in APIs using protocols like REST where there are clear actions for retrieving (GET, HEAD, etc.) and changing (POST, PUT, PATCH, etc.) state. We can implement independent paths for reads and writes, which helps in the service's organization.

CQRS, however, is often associated to an architectural point of view instead of just locally in code. We can take it one step further and segregate commands and queries into different components. We can have two different services, one that only handles commands or changes in state and another that only handles queries that only retrieve data without changing it. Figure 4-13 illustrates how the example we discussed before with the product service would look like when applying CQRS.

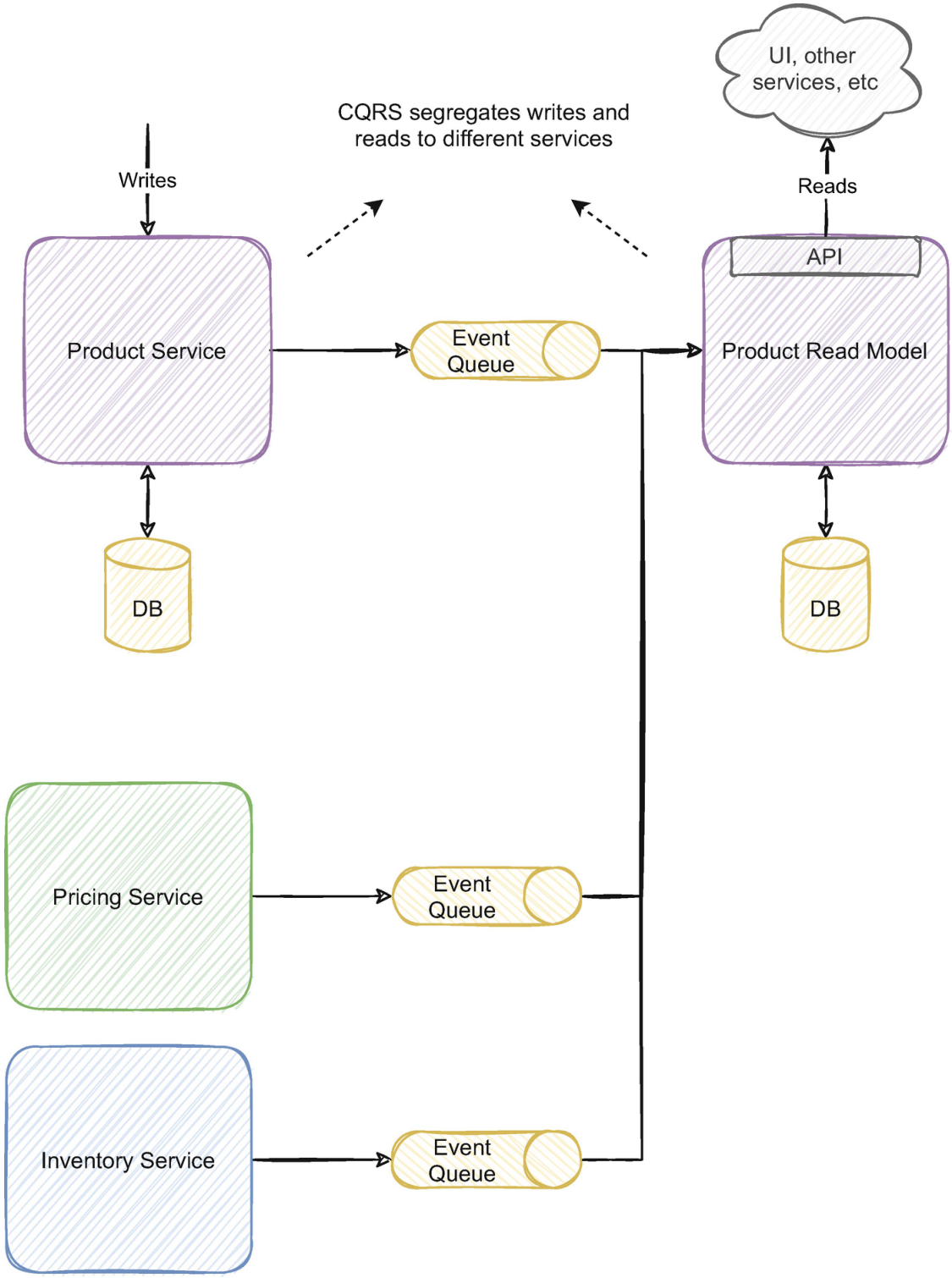

Figure 4-13 When applying CQRS at an architectural level, we can have separate services for reads and writes

When applying CQRS at an architectural level, we segregate the writes to one independent service and the reads to another independent service. The product service handles writes that can be done from an API or a command queue. The product read model handles reads and exposes the data through an API. The read model also handles the events from pricing and inventory service to enrich its model and be able to do the complex searches we mentioned earlier. But what do we benefit from this?

Having separate read and write models allows us to specialize each model for each purpose. The objects we return on reads, traditionally called DTOs (data transfer objects), are often modeled to the use case that needs it, for example, a UI. That model can be substantially different from the conceptual domain model, which is often modeled by the domain concept (the aggregate in DDD) and bearing in mind performance and transactional concerns. Often the query requirements end up influencing the domain model design negatively. Usually, it is also required mapping between the two, which can be more or less complex depending on the differences between them. Often having the same model for both ends up doing neither one well. Having a separate model for each, we remove a considerable part of that hassle.

Earlier in this chapter, we discussed the need to abandon the third normal form due to the data's denormalization. The third normal form minimizes data duplication which actually benefits writes. Since we have two separate models, we could apply the third normal form to the write side and denormalize the data on the read side.

Having two separate models with two different databases also allows us to optimize the model and the technology for each purpose. The write side could have data modeling optimized for writes, for example, event sourcing (we will detail event sourcing next) which is basically an append log enabling minimal locking with a technology optimized for writes. We can denormalize the model on the read side and use a technology built for fast complex searches (e.g., ElasticSearch).

New features and new query requirements can also be easier to implement. For example, if we need to launch a new UI with a different model and query needs, we would only change the read model service. That change wouldn't impact the service that handles the writes and has the domain logic.

Consistency can also be a factor with this approach. Usually, the write side has stronger consistency requirements due to the domain validations and the need to write consistent data to the database. The read side can often be more lenient to weaker consistency guarantees like eventual consistency. Besides this, most applications tend to exhibit a read-inten-

sive behavior; the number of reads vastly surpasses the number of writes. Having separate services for each allows us to optimize and scale each side independently.

When to Use CQRS?

CQRS is not a silver bullet, nor should it be applied to every use case. It implies one more moving piece and increases the complexity of the solution. Having the writes separated from the reads with an event queue in between means the read side is also eventually consistent if it wasn't already. So when should we use CQRS?

Using CQRS has unsurprisingly similar motivations to adopting an event-driven architecture. Typically, simple and straightforward domains usually don't benefit much from it. There's little benefit in dividing something that's small and simple to begin with. Often domains that aren't close to the business core and don't have enough complexity or change too often don't benefit much from it.

A simple domain with a low scale that often doesn't change and needs strong consistency guarantees is a good candidate to not apply CQRS. I experienced simple services with small amounts of data where the teams applied CQRS and it only added confusion to both the architecture and to the business. Simple services often are better off with a traditional CRUD approach.

It's important when deciding to adopt CQRS to have a strong reason to do so. Some of the benefits we listed before can be arguably strong reasons, for example, if we have a service where the reads largely surpass the writes and we have the need to scale differentially. Having a conceptual write model substantially different from a complex read model might also be a good reason.

Using event-driven architectures, you will likely have some sort of CQRS or at least a denormalized model that aggregates information from different places. For example, the query needs in the example we discussed with the product service, we needed to merge data from various sources to provide meaningful searches to the business. When we need to

build a similar read model, a good approach is to apply what we learn with CQRS, to understand if it makes sense to build it in a separate service and whether the benefits of CQRS are relevant for our use case. Figures 4-12 and 4-13 show two different approaches; we could ask some questions to help us understand if CQRS makes sense. For example, by merging everything in the same service, are we polluting the product domain model? Is the number of events from the other sources sufficiently disparate to justify independent scaling? Would the searches benefit from a different technology we are currently using in the product service?

These are just some questions we should ask ourselves when applying CQRS. Overall, it has high synergy with event-driven architectures, and it is valuable in some use cases. But we should always question if the benefits outweigh the complexity overhead.

4.5.2 The Different Flavors of CQRS

In the previous section, we discussed what CQRS is and how to apply it. We discussed its advantages and when we should use it. In the example we discussed, we created two different services with two distinct databases. This solution isn't the only way to apply CQRS, and there are some intermediate alternatives that we will discuss in this section.

CQRS is fundamentally about segregating the reads from the writes. In the example in Figure 4-13, we created one independent service for writes and another one for reads. This approach is the most flexible and the most complex. It makes a lot of sense in event-driven architectures because most of the drawbacks with that approach are already incorporated in the architecture. For example, eventual consistency is often a byproduct of most event-driven services and needs to be tackled consistently (no pun intended). The concurrency and ordering challenges of events, which might be hard to tackle in an architecture not oriented to them, are already a reality, and we likely already have strategies to address them.

However, it isn't the only approach. CQRS doesn't mandate we have a separate database for each service, for example. Figure 4-14 illustrates an alternative

implementation of CQRS in the same example of the product service we discussed before.

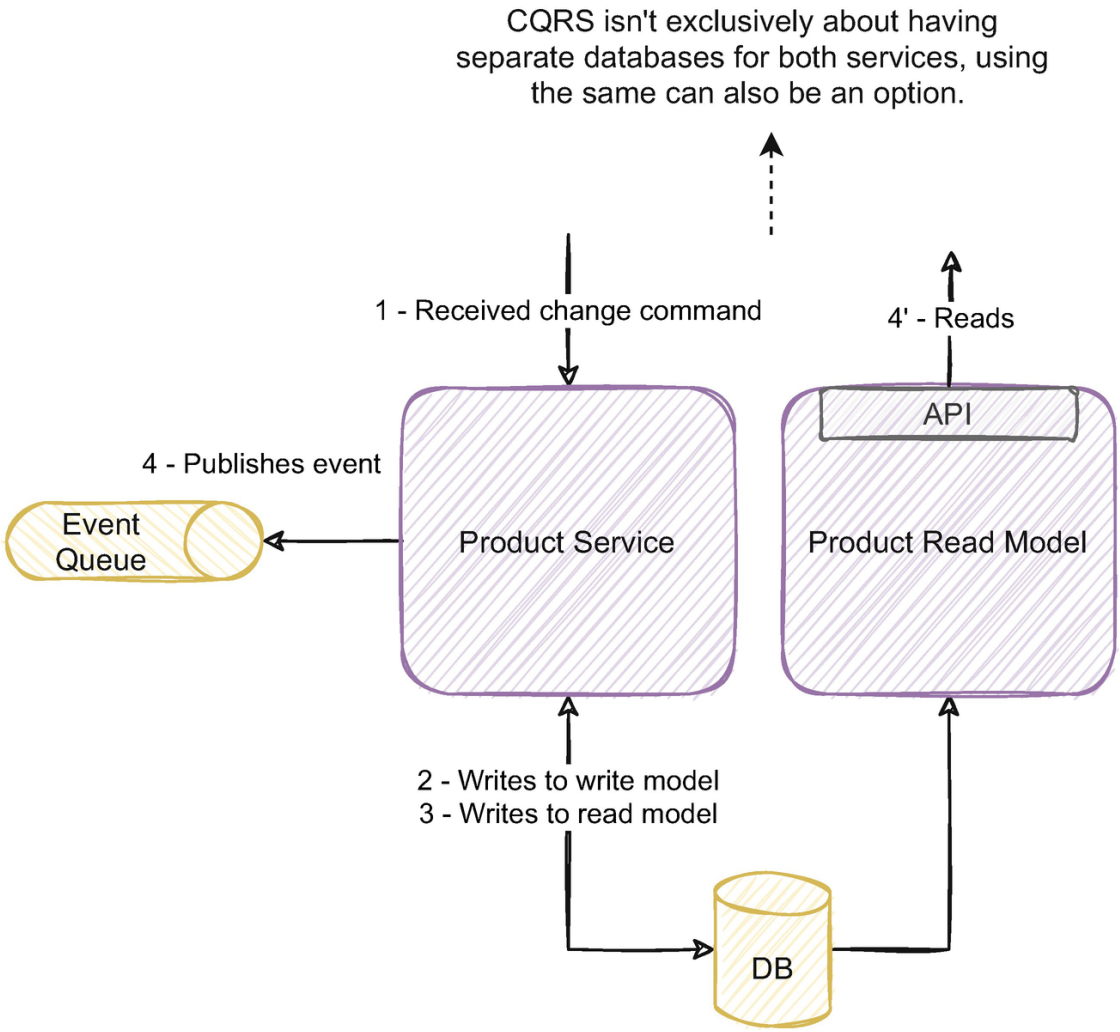

Figure 4-14 We can use different approaches to apply CQRS, with different tradeoffs

Every time I see the same database shared by two independent services, I get post-traumatic stress flashbacks. But there are advantages to applying the pattern this way. A troublesome issue by having two separate databases is the associated eventual consistency. We will discuss strategies to deal with eventual consistency in Chapter 5, and as we discussed before, most querying needs tend to be lenient weaker consistency guarantees. The weaker consistency guarantees often hold especially true for UIs. Eventual consistency isn't new and wasn't introduced by events. Caches are a form of eventual consistency, and we apply it thoroughly. However, when another service needs to query an eventual consistency model and get stale data, it will do its business logic based on that stale data, leading to erroneous results.

For example, in Figure 4-13, there was a pricing service also listening to the events being published by the product service and had the need to ask for additional information from the read model API every time it received an event. If the product read model didn't yet process the same event the pricing service is processing, it would return stale data. The data returned to the pricing service wouldn't match the event the pricing service received.

With the approach depicted in Figure 4-14, we can update both the write and read model before publishing the event; this would solve the inconsistency between the event and the API response. With this approach, we can still scale the read and write side independently (although not the database). We do have to live with the same database in both services, which can be a challenge to manage schema changes. This approach might be a good option when there is a close relationship between the write and read model. If there is a situation where a read model listens to events from different sources (like Figure 4-13), we might benefit more from a more flexible option and more substantial segregation.

There are other options; we could not have two different services, having only one and having strict segregation of reads and writes only in code. On the other end of the spectrum, we can have a third service that handles writes to the read model (one service that writes to the write model, one that writes to the read model, and one that reads from the read model). In the last option, I seldom saw the benefits outweigh the drawbacks.

As with most solutions, it's all about the tradeoffs. The main goal is for you to have these tools in your toolbox and be able to decide accordingly – whether it's more fitting having a highly flexible highly complex solution or a less flexible less complex one or not using CQRS at all.

4.5.3 When and How to Use Event Sourcing

A typical pattern usually mentioned along with CQRS and DDD is event sourcing. Event sourcing is also an interesting pattern to apply in event-driven architectures since it has high synergy with the event-driven

mindset. This subsection will detail event sourcing, its benefits, and how and when to apply it.

Let's look at the same example we discussed until now with the product service. When stored in a relational database, the product information would probably look similar to what's depicted in Figure 4-15.

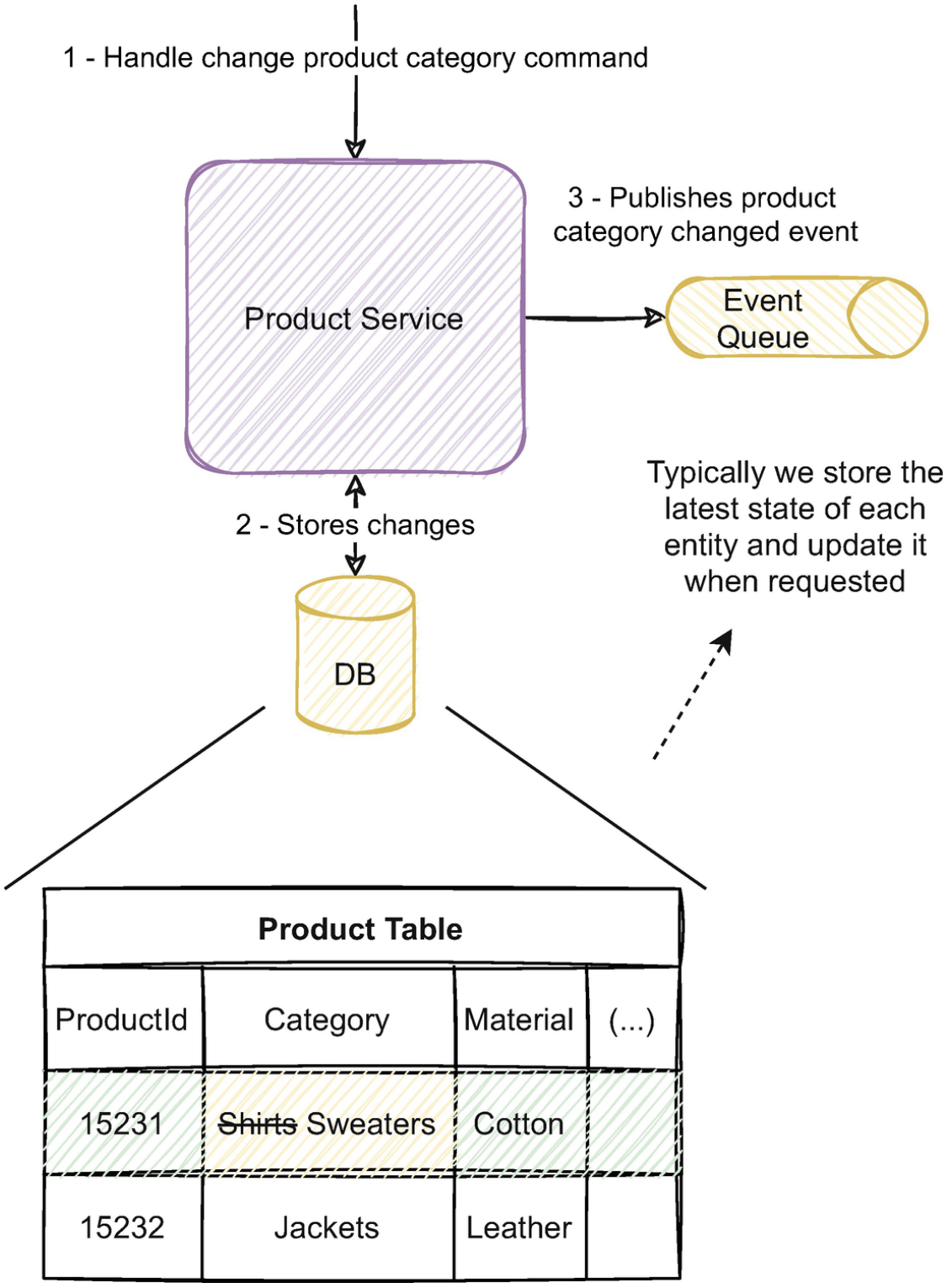

Figure 4-15 Typical state storage in a relational database

The product service handles a command to change an entity, in this example product 15231, changes the information in the database, and publishes an event signaling the product changed its category. The product table stores the latest information about each product. This is a common way to store data; we store the latest state and manipulate it when someone requests a change.

Event sourcing proposes an alternative way to store the entity's data. Instead of saving the latest state, an entity is the history of changes that generated that state. Conceptually the entity no longer is materialized with the most recent value of each property; instead, an entity is a stream of immutable events. If we apply all events of the stream, we can obtain the latest state.

For example, to store the product entity, instead of persisting each property's latest values, we would persist the events that signal each change. This is illustrated in Figure 4-16.

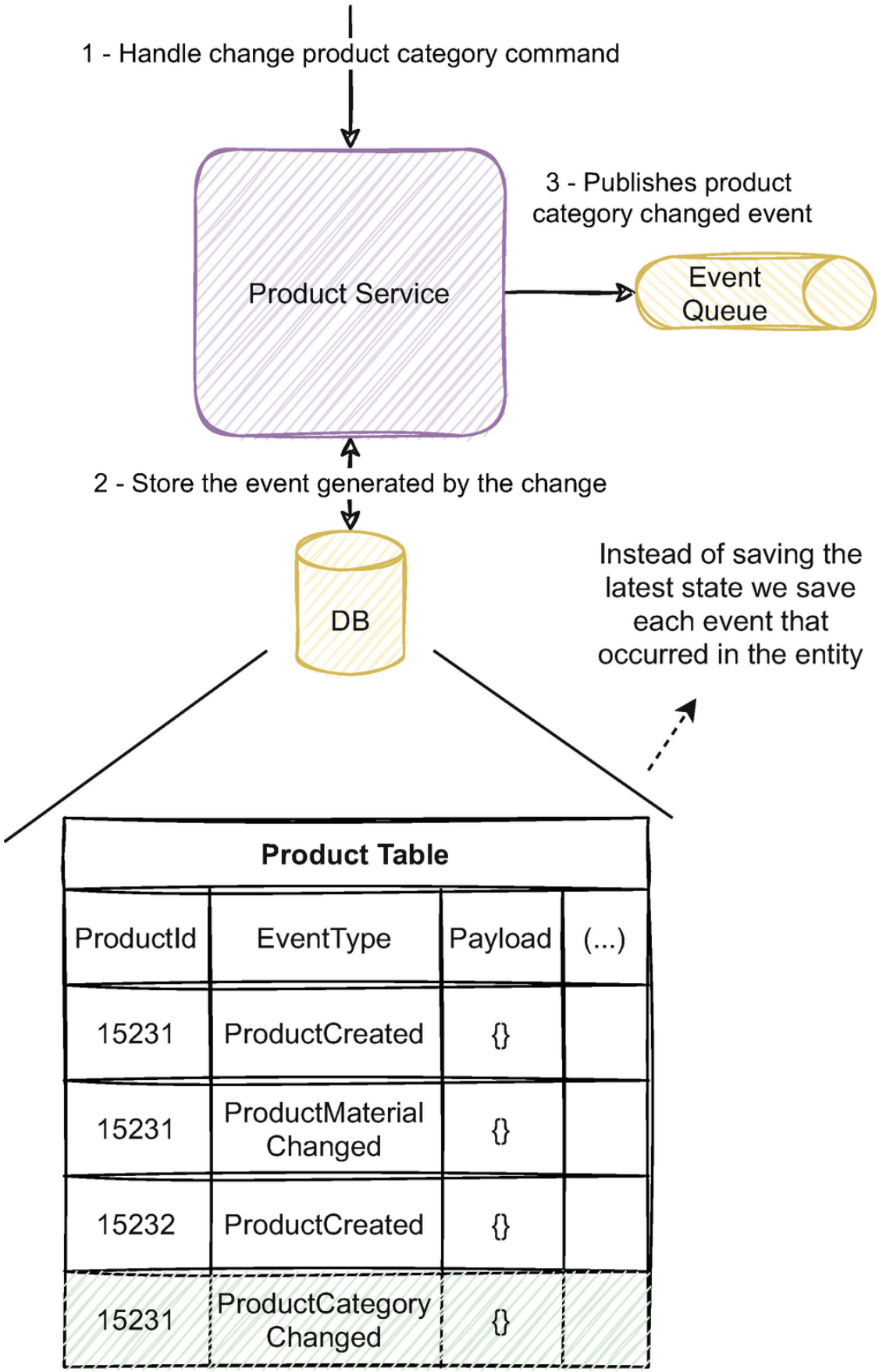

Figure 4-16 Instead of saving the latest state of each product, we store the events generated by the change commands

Instead of changing the product's category to the new one, the product service stores the category changed event that was generated by the

change. The database stores the event type and the payload of the events. Instead of being a representation with the id, material, category, and so on, the products are the stream of events that occurred until now. The latest state of the product can be generated by applying all events in the stream.

At first sight, this sounds like a terrible idea, right? How are we supposed to query the data? For example, how can we obtain all products with category jackets? All we have persisted in the database is the history of events. To filter them, we would need to apply all events for each product to determine the latest state and then query.

That's one of the reasons event sourcing has high synergy with CQRS. As we discussed in Subsection 4.5.1, if we separate the write model from the read model, we can have substantially different models for each. We can use event sourcing on the write side and use the events to persist the latest state on the read side. By doing so, we can have an optimized model to query the data and an optimized model to write data.

As we discussed, events represent something that happened in the past and are immutable. The event stream basically functions as an append log. Write operations are often faster than update or delete operations; due to this, using event sourcing in the write model can often benefit from better performance.

An important consideration when building an event-driven architecture is to make events the source of truth. Event sourcing fully embodies this concept as events are the entities. The event stream can also provide a comprehensive history of the entities' state or be useful for debugging and auditing purposes. But the highest value is the ability to retain the original data and rebuild different views of the data for current and future use cases. We have the query requirements we need today, but we can rebuild different views of the data to future use cases by having the full history of the event stream.

If you are using a relational database, there is also a performance and conceptual gain in using event sourcing. There is an impedance mismatch (there's an article[7] by Scott Ambler further detailing this, and Greg Young

explores this concept thoroughly in his CQRS documents[8]) between the OO (object-oriented) model and the relational world. How entities are modeled is substantially different between the two. This difference usually has performance impacts and additional complexity to reason with the models; complexity ORMs (object-relational mappers) tend to abstract. Using event sourcing simplifies these issues since the event is the same in both.

Figure 4-16 illustrates events being stored in the database and published in the event queue. With durable message brokers, we can further improve this topology by not having a database. Figure 4-17 illustrates this situation with the same example we used before.

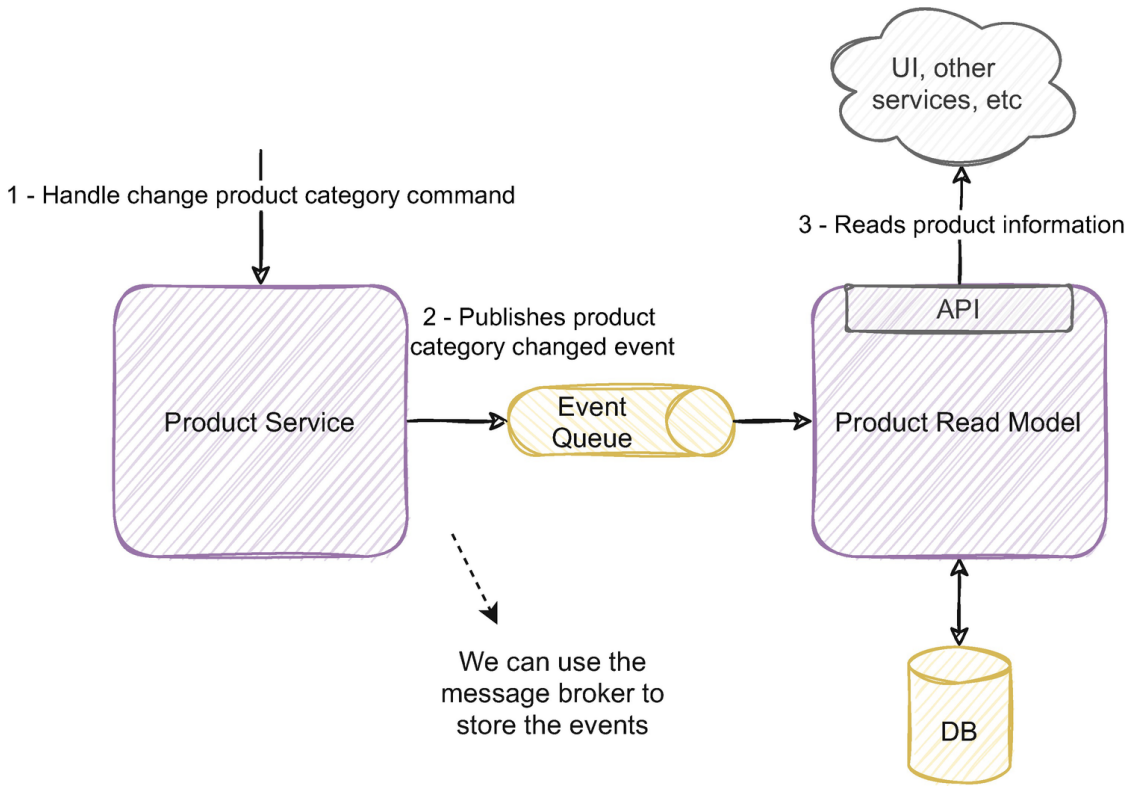

Figure 4-17 The same example as before but using the event broker to store the events

In the Figure 4-16 example, we had the events stored in the database and in the event queue. Notice in Figure 4-17 that the product service, which is responsible for handling the commands and applying the business logic, no longer stores the events in an internal database. Instead, it publishes the events to the message broker, and the events are stored there. This way, the events no longer are in two different places, benefiting from the infrastructural gains and the associated issues of guaranteeing both places are exactly the same. Also, storing and publishing events

imply we need to ensure that both operations happen or fail together; otherwise, the system will become inconsistent. By only having one operation, this concern is greatly simplified. We also fully embrace a single event stream as the source of truth.

Storing events only in the message queue begs the question of whether we can fully trust a message broker to store the core business information. A few years ago, this kind of option would send a chill down my spine. But the rise of event streaming and the need to do it at a large scale certainly highlighted the concern and paved the way to do it reliably. Are durable message brokers a database? They certainly don't have the query capabilities we are used to with traditional databases, but they can afford at least as strong consistency guarantees as distributed databases. Martin Kleppmann has a very interesting keynote[9] about this subject with Kafka, which we recommend to further the topic.

In Figure 4-17, we also combined CQRS with event sourcing. This way, the product read model builds a denormalized projection of the product data optimized for querying. Using a persisted event stream in the write model, we can also rebuild the same or new models on the read model when needed.

4.5.4 Concerns and When to Use Event Sourcing

The main concern of event sourcing is the ability to query the data. This concern is mostly mitigated by using the read model and building projections to answer those querying needs. However, the read model is eventually consistent. When business logic requires to query the data, doing so in an eventually consistent read model can be hard to deal with. For example, if the product service had a business rule that only 25% of the products can have a price above 400$, it would have to validate that rule every time a product is created or its price changed. It would be hard or infeasible to do that query on the write model. We could query the read model, but the inherent eventual consistency could generate erroneous results. We further detail how to deal with eventual consistency in Chapter 5, but it always has a complexity increase.

Saving every single event of every single entity can also be impactful both cost-wise and performance-wise. This solution often requires retention strategies and the usage of snapshots. Although feasible and retention policies are probably already a concern on non-event sourced systems, snapshots require dedicated implementations and add to the system's overall complexity.

We don't need and shouldn't embrace event sourcing in every single service in the architecture. Usually, services with complex querying requirements might benefit from event sourcing and CQRS. Services that deal with core domain concepts and where the business might need to extract different projections from that information in the future also might be a good indicator. Having strong audit requirements is also a good fit since the event stream is the source of truth it can provide a detailed audit. Simple isolated services are likely not to be a good fit since they won't likely benefit from the associated complexity.

In event-driven architectures with durable message brokers, we already face many of these concerns since we have persisted event streams. Using event sourcing is stepping up a notch to model our entities with event streams and fully embrace them as the source of truth.

4.5.5 Using Command Sourcing and Its Applicability

A very similar pattern related to event sourcing is command sourcing. Unsuspectingly, command sourcing uses the same approach event sourcing uses, but instead of persisting events, we persist commands. This subsection will discuss command sourcing and apply it in the same example we discussed previously.

In Subsection 4.5.3, we discussed how to apply event sourcing in the example with the product service. Using the same example, we could apply command sourcing by saving the commands reaching the service. Figure 4-18 illustrates how we could apply it.

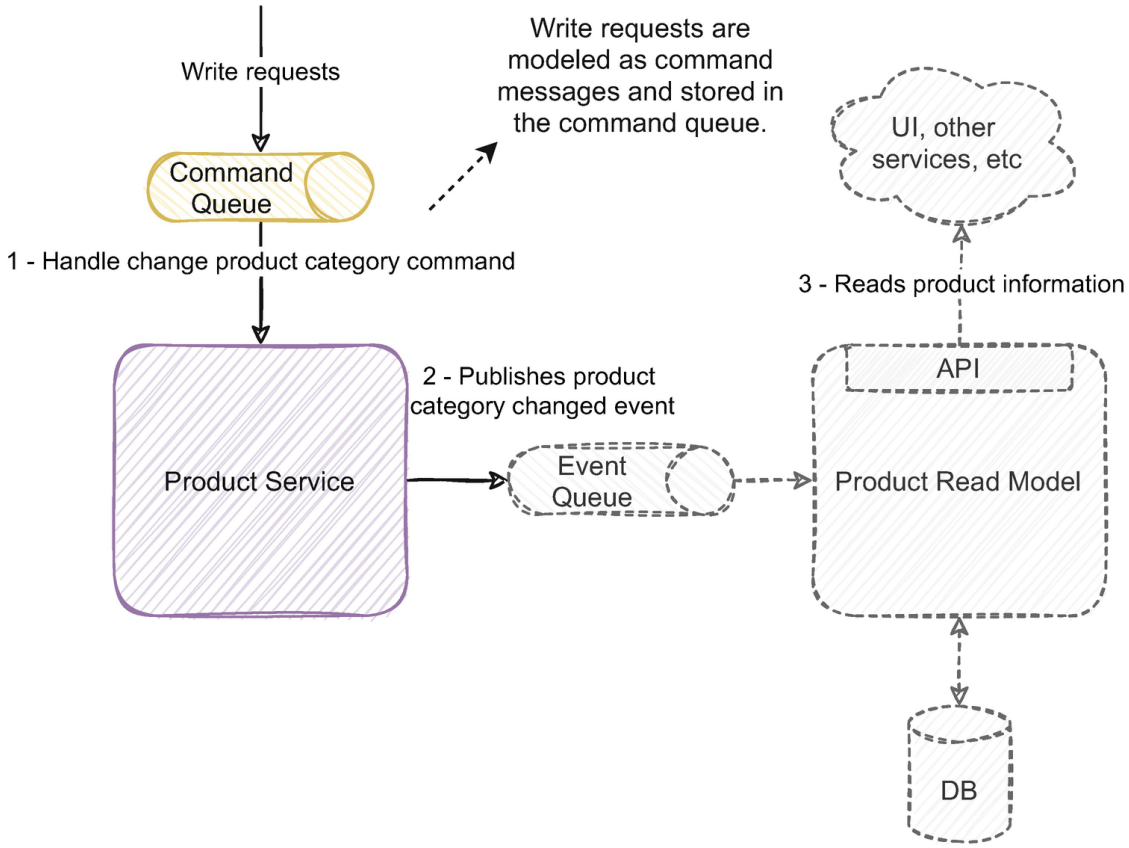

Figure 4-18 Command sourcing applied to the same example with the product service

Write requests are modeled as commands and sent to a command queue. The commands can be stored in the queue, as discussed previously with durable message brokers, or stored in the database. The product service consumes the commands from the queue and generates events. The events are published to an event queue and consumed by the read model where the data can be queried the same way we discussed before.

Command sourcing adds an additional layer of complexity and asynchronism to the flow. Write requests to the service can be refused if they fail the domain validations. If the request is synchronous, it is easy to return the feedback to the application issuing request. However, if it sits in a queue, it will be processed later. Returning feedback about validation errors might be troublesome. The concern is not because it is asynchronous, but the application will have to handle asynchronous feedback about failures to accept the commands and handle the regular events to understand when the request is applied. Although feasible, by handling the events and differentiating them, we might be walking very close to the line that separates a good solution from over engineering.

Command sourcing saves the user's original request before the application manipulates it, applies its logic, and transforms it into an event. It is often useful when the service receiving the commands has complex logic or intricate algorithms. Saving the original request allows us to regenerate the state if we introduce a bug in the service. It might also be useful if we want to try different versions of the algorithm and understand the results.

If there is an imperative reason to save the request precisely as the user submitted it, then command sourcing might be a good option. However, depending on the application needs more often than not, event sourcing is enough, and the complexity overhead doesn't pay off. In event-driven architectures, especially the parts using choreography, events are the glue that connects the different services throughout the flow. Commands are used less often in the places that we use commands; command sourcing can be a useful pattern; however, be aware of the complexity overhead and weigh whether it is worth it.

4.6 Building Multiple Read Models in Event-Driven Microservice Architectures

In Section 4.5, we discussed the difficulties of satisfying query requirements on data that is dispersed throughout several distributed microservices. We discussed that we could build a denormalized read model by handling events from different sources. This section will detail how to do it and how we can apply some of the concepts we discussed previously.

Let's use the practical example we worked with in Section 4.5. We have the requirement to list all products with stock and sort them by descending price. The relevant information is in three different services, the product, inventory, and pricing service. As we discussed, to be able to sustainably do that query, we could use an independent service to handle the events from each service and build a denormalized read model optimized for it. This situation is illustrated in Figure 4-19.

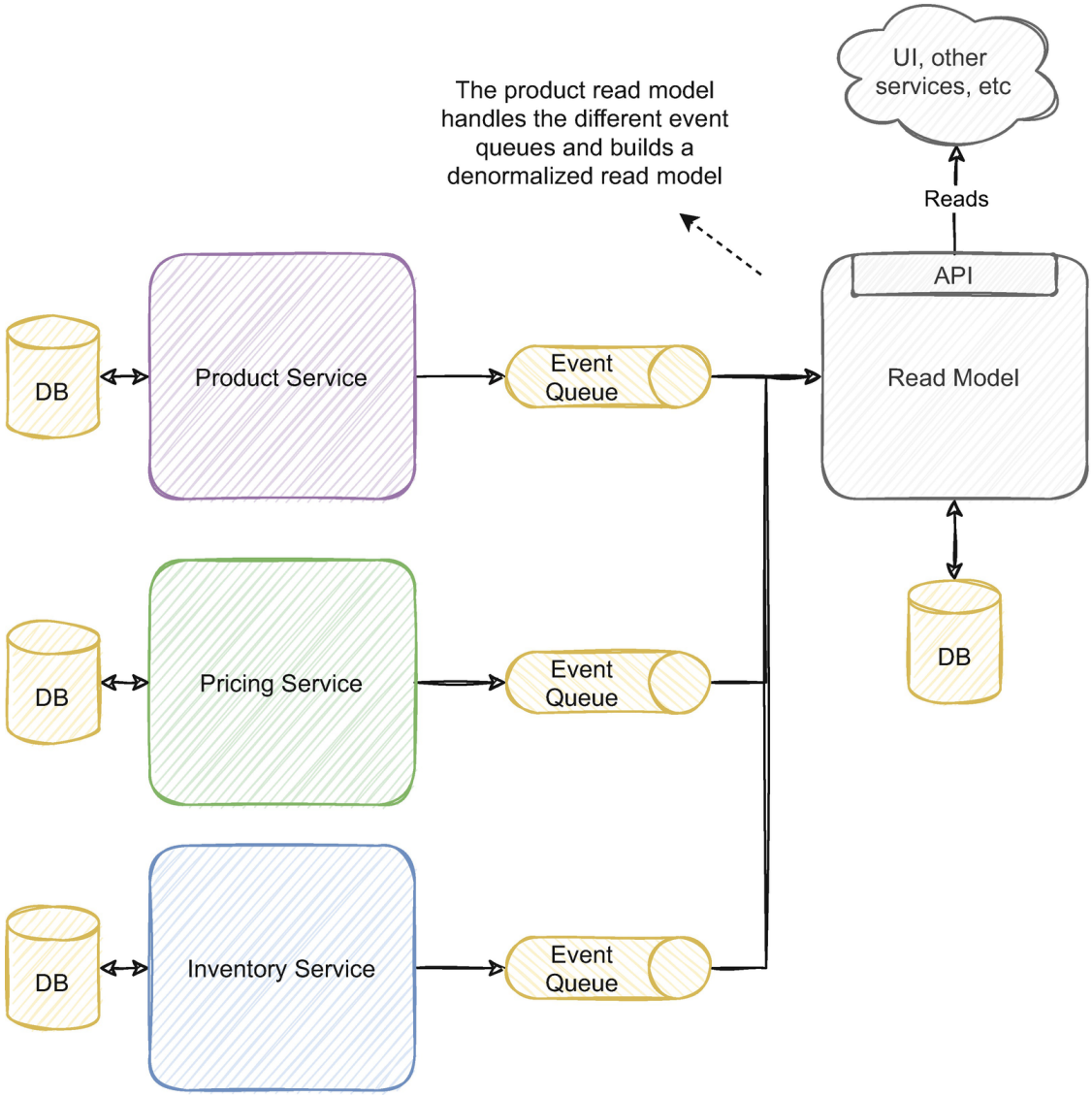

Figure 4-19 A read model handles events from different services and builds a denormalized model

In event-driven architectures, and often in microservice architectures also, these kinds of queries are often a common challenge (many of them similar to the ones raised when we discussed the API composition pattern). They are often solved by opting to build this denormalized read model. Event-driven architectures provide a way to do this by sharing information through events and offer a practical way to make the data readily available. The event streams provide a decoupled way to process data and transform it into something else, dedicated to a different purpose. They also do it in a continuous manner and in real time (as long as the subscriber is able to keep up with the throughput, which shouldn't be a problem since they are horizontally scalable). The associated decoupling of the services also guarantees that they don't impact each other by doing so.

But what kind of events would be best to build that denormalized view? As you'll recall, in Section 3.1, we discussed the differences between events and documents. Let's go through this practical example and discuss the tradeoffs.

As we discussed in Chapter 3, partial events are relevant when a service needs to react to that specific change. Read models typically have little or no business logic, as the business rules should be in the service that owns the domain and is the source of truth. We also discussed that documents are often relevant when the consumer uses most of the entity's data and is only interested in the most recent one. From this perspective, it would make sense to use documents to feed the read model. For example, Listing 4-1 illustrates a possible document the product service could publish.

```
ProductDocument
{
    Id: 152397,
    Brand: "Nike",
    Category: "Shirt",
    Material: "Cotton",
    Season: "Spring/Summer",
    Sizes: ["XS", "S", "M", "L", "XL"],
    Colors: ["Black", "Gray"],
    CreatedAt: "2021-01-30T11:41:21.442Z"
}
```

Listing 4-1 ProductDocument schema definition

If we had a partial event for each property, the read model would have to handle possibly six different events with different logic. A single document with all the information dramatically simplifies the effort to update the read model. It also reduced the number of events we have to publish and consume. For example, if the user edits several fields in one action, only one product document needs to be published, while if we had several partial events, we would need to publish one for each changed property.

An interesting solution is to combine this approach with Kafka's compacted topics. As we discussed in Chapter 3, when using compacted topics, Kafka removes older messages that share the same partition key. Figure 4-20 illustrates how this mechanic works.

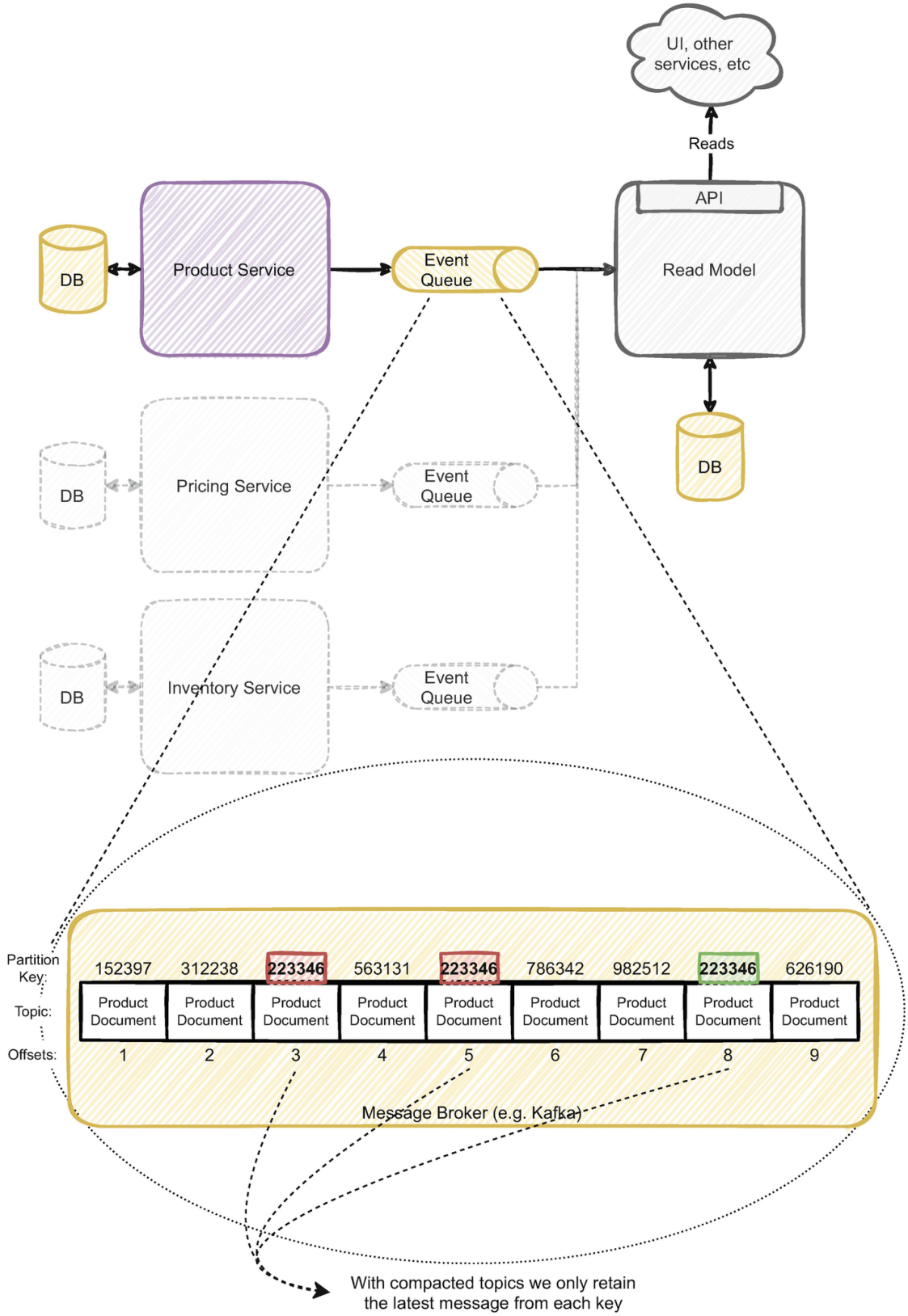

Figure 4-20 Removal of older messages with Kafka compacted topics

Notice that this is the same diagram we discussed earlier in Figure 4-19 but highlights the event queue's inner workings. We have a queue with only product documents, since we decided to publish just the document and don't use partial events. The numbers above the documents are the partition keys, which in this case translate to the product id, and the numbers below are the offsets. When the product service produces messages,

it uses the product id as the partition key. Kafka internally uses this key to distribute the messages between different partitions (a concept similar to sharding, we will further detail how this works in Chapter 6).

When using compacted topics, Kafka removes the older messages with the same partition key. In the example in Figure 4-20, we have three documents with the same partition key (223346). The product service generated the three documents because the users changed the same product three times, so the product service published three documents informing each change. The two documents with offsets 3 and 5 are removed since they are older, and we only retain in the queue the latest one, with offset 8.

Compacted topics only remove older messages from repeated partition keys and always keep the most recent message per partition key. The product document we designed gives a comprehensive view of the product and its information. By combining both, the compacted topic acts as a product database but built in a way that is ready to be shared with other applications, and in a streaming fashion, new updates will be published as new documents at the end of the topic.

How could we build a similar denormalized model in a traditional microservice application using synchronous requests? Perhaps the read model would have to poll the product, pricing, and inventory service and fetch information about each product. This alternative would create a direct dependency between the read model and the other services, which would make it susceptible to cascading failures. It would also be less performant since the read model would have to request information even when no update was made. The time between editing information and being available in the read model (the inconsistency window) would be larger since it would be based on a specific interval rather than reacting to real-time changes. Scaling is also a challenge. The more requests we do to the services, the more the strain on the database; using this solution frequently might eventually impact the service responding to the requests, which begs the question of whether the solution is genuinely horizontally scalable.

Using an event queue and a streaming solution, we solve these problems in an organic way; changes are published and the services react to them. They are fully decoupled and don't need or impact each other directly. Adding more instances to the read model service, for example, would never impact the product service.

Rebuilding the model in case of an issue is also straightforward. Imagine if we introduce a bug in the read model service and the data is corrupted. If we need to rebuild the whole model, we can simply consume the topic from the beginning. Compacted topics retain the latest document from each product; by reading it from the beginning, we will consume the data from each product. There is no need for an ad hoc process or specific built application to migrate the data; neither is there the need for downtime. The code to rebuild the model is the same as the regular routine events. If we need to do it faster, we can add more service instances to consume the events more quickly during the rebuild.

If we receive a different search requirement from the business that our current model isn't optimized to fulfill, we can use the same strategy to generate the new model. Suppose we need to denormalize a different projection of the product data; we have the product information in the queue. In that case, we can use the same strategy to read it from the beginning and build that new projection.

Overall, this solution follows the same principles we have been discussing throughout this chapter; when data reaches a given scale, it often requires denormalized projections. Approaching the solution in an event-driven mindset provides us with the flexibility to build it according to the search requirements in a scalable and more reliable way.

4.7 The Pitfall of Microservice Spaghetti Architectures and How to Avoid It

In this chapter and the previous ones, we discussed how to transition to a microservice event-driven architecture, and we discussed how to build event-driven services and apply structural patterns. As we grow the microservice ecosystem by adding new functionality or creating more fine-

grained services, the overall system's complexity continually increases. Besides all the challenges distributed architectures have, continuously adding more moving parts to the system can become hard to read and understand the overall picture. In this section, we will discuss this challenge, its impacts, and how to address it.

In a complex network of microservices, you often fail to find the correct flow. The same way we often struggle to understand the flow and logic of a single application's spaghetti code, a complex architecture can be even more challenging to read. When a business process spans several services, analyzing the code of several different services to understand the high-level flow can be a real detective's work.

Yuri Shkuro from Uber has a very interesting presentation[10] about tracing 2000+ services. The service network generated by Jaeger shows this difficulty and the associated complexity of such a high number of different components. Although the difficulty of understanding a complex web of distributed components is a drawback of distributed architectures and something we have to live with, we can take several actions that help us reason with that architecture more sustainably.

As discussed in Chapter 1, a drawback of monoliths is that they often have the right conditions to create spaghetti code without the right mindset and discipline in place. Microservices solve that since each service is physically separated from one another. In truth, they arguably delegate the challenge to a higher level, to the microservice organization and higher-level architecture. It certainly has many advantages; for example, it's easier for the teams working with the services to organize code and build new features. However, it's also easier for everyone to lose track of the overall flow and how the services impact each other, often creating complex corner cases. There are many resources and documented strategies to address spaghetti code. However, addressing spaghetti architectures often falls into a more dubious and more complex scope.

4.7.1 Domain Segregation and Clear Boundaries

In Chapter 3, we discussed DDD as a way to organize service boundaries. This organization can also be an excellent way to manage the microser-

vice complexity. As we discussed in Chapter 3, services organized according to their domains are likely to be more stable than other types of organizations or not having one at all. They often prevent the impacts of new development to ripple through several components throughout the architecture.

Often in event-driven architectures, the domain's flow and the business processes are translated into the message flow between the several components. The highly evolutionary nature of these architectures also paves the way for them to change seamlessly. Losing track of the domain flow is easy, and understanding it is often tricky because it requires analyzing the message flow between several components.

Understanding the overall flow is often facilitated when services are clustered according to their domains. For example, in the practical example we discussed when detailing Sagas in Subsection 4.1.3, for fulfilling an order, we know we have to do a series of steps, for example, saving the order information, validating and updating stock, calculating pricing fees, etc. If services are organized using domain segregation, it's easier to map the business process flow to the services that implement it. Each service, or set of services, belongs to a bounded context related to a business concept. Even though there might be many services, each bounded context communicates to each other the same way the business process conceptually does, and the message flow advances the same way. This way, it's easier to reason with the architecture and the interactions between the services.

Another essential principle is to build clear boundaries across domains. One typically good practice in software engineering is to use dependency inversion (the D in SOLID[11]). It states that the code should depend upon abstractions rather than concrete implementations. Applying this to a high-level architecture, one boundary should access the other through the public interfaces, never by the internal service's implementations or endpoints.

You might recall when we discussed modular monoliths in Chapter 1. Modular monoliths are typically built by defining these boundaries and maintaining strict isolation between them. Shopify even built[12] a way to

programmatically enforce these boundaries by validating code that doesn't access the public interfaces. It's easier to bypass these boundaries in a monolith since the code is centralized in a single solution.

However, in a complex microservice architecture, we can struggle with the same issue by having a chaotic network of dependencies between the services. Domain organization and strict boundaries mitigate this issue. Boundaries should be explicit on the endpoints they expose and clarify what functionalities are public and which ones aren't (similar as they would be in a single code solution). This encapsulation helps contain the dependencies and lowers the likelihood of breaking contracts. A good approach is often guaranteeing the boundary has exclusive ownership of its domain's data and has clean dedicated endpoints that expose the boundaries' functionalities.

Overall, having domain segregation and enforcing the separation of concerns between bounded contexts can be useful in containing the architecture's complexity and a sustainable way to evolve the architecture without losing track of the high-level flow.

4.7.2 Context Maps

The patterns we discussed, like CQRS and event sourcing, aren't applicable in every use case, but DDD and its techniques have essential benefits that we can often use. DDD highlights the need to understand the system's domains and bounded contexts and their relationships. Its practices can often help us understand them even if we don't apply the more complex patterns like CQRS.

Context maps are typically mentioned in DDD, and they describe the existing bounded contexts, their relationships, and interactions in the form of a diagram or simply text. They can depict the higher-level relationships, dependencies, and flow of messages between the different bounded contexts. They are often similar to the diagrams we have been using throughout the book; an example is depicted in Figure 4-21.

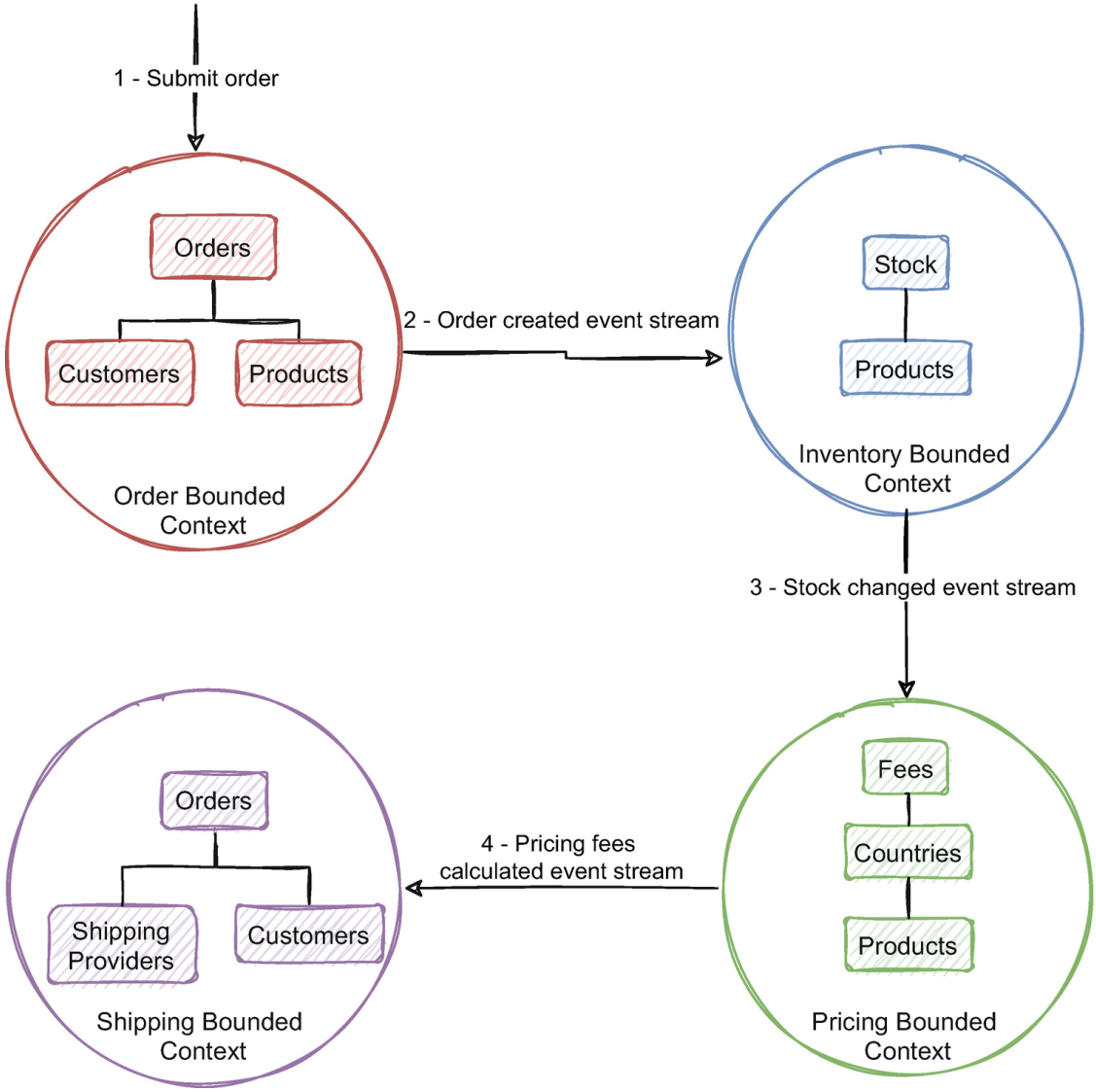

Figure 4-21 Example of a possible context map for the eCommerce platform we have been discussing

This example also details the entities in each bounded context. Also, an interesting note is the product entity, which is present in several bounded contexts but might have different information on each, depending on what is relevant. We can start by only drawing out the relations between the bounded contexts and later add more information like domains and entities. The details about relationships and their dependencies are often described in text or in a table that follows the diagram.

Context maps help to reason with the system, understand each boundary, and highlight where the boundaries share data. They also help to picture how the most important parts of the system relate to each other. In event-driven architectures, we can enrich the context map with the external event streams exposed and handled by the different bounded contexts. This approach can help provide a reference to reason and organize

the architecture without losing track of the relationships between the services.

4.7.3 Distributed Tracing

In event-driven architectures, there is often no direct communication between services; this and the high decoupling between components can make the data flow hard to understand. Especially when using choreography, it is easy to lose the high-level flow of the events. Distributed tracing can be useful to tackle these challenges and help to map and understand these flows.

Distributed tracing collects and records services and operations that are in the scope of the events. That data can then be used to be searched and visualized. It can be a valuable method to reason with a distributed system, especially a highly decoupled one driven by events. It can also help to understand the relationships between the services.

Several tools can help us implement and record tracing. For example, we can use Zipkin along with Kafka's interceptor API[13] to record traces for every event produced and consumed. Another interesting example is using OpenTracing along with Jaeger.[14] OpenTracing can be used to add correlation data to the event's headers and use them in Jaeger to have a high-level visualization.

Distributed tracing can be a valuable tool to visualize how services interact, the flow of the data, and a high-level perception of how architecture evolved throughout time. We can use that high-level view provided by distributed tracing to organize and reason with the architecture in a more sustainable way. We can't improve what we don't measure; if we have that information and if the architecture has a chaotic network of dependencies, we can use that information to design a strategy to organize and make sense of the architecture's functionality.

4.8 Summary

- Transactional consistency is a complex challenge in distributed systems. Distributed transactions and two-phase commit protocols are an alternative, but a very limited one often raising more issues than it solves.
- Sagas are a sequence of individual operations to manage a long-running process. We can use them to divide a long-running or traditional single database transaction into smaller ones, being more suitable for a distributed environment. Two common patterns of implementing Sagas are orchestration and choreography.
- Orchestration uses a primary component to manage the steps of the process. It is often valuable in complex processes that need a central supervisor.
- Services using choreography react to each other to complete the Saga's sequence of steps. Each service reacts to the changes happening in the ecosystem to accomplish its tasks. Choreography tends to be a typical pattern in event-driven architectures, and it synergizes well with its mindset.
- We can combine orchestration and choreography as we see fit. Choreography can be applied more commonly and in higher-level processes. We can use orchestration on a more limited scope and specific use cases.
- In event-driven architectures, having a set of independent services, each owning its domain's data, can pose a challenge to getting an aggregated view of the data. API composition can be a possible solution, albeit a very limited one.
- We can apply CQRS to segregate writes from reads. This segregation allows an optimized model and approach for each. CQRS isn't limited to segregating databases for reads and writes; there are other intermediate patterns that we can use.
- Event sourcing is a valuable pattern that synergizes well with event-driven architectures. It is important to understand the concerns and advantages of event sourcing as they might be useful in some use cases but also produce some complex limitations.
- Command sourcing relates to event sourcing and can also be a useful pattern when it is essential to save the requests exactly as the user submitted them. As with event sourcing, it is important to weigh its benefits and apply it only when it outweighs its concerns.

- Documents and Kafka's compacted topics can be a good approach to build several read models.
- Continuously adding more moving parts to the system can become hard to read and understand the overall picture. Domain segregation, clear boundaries, context maps, and distributed tracing can be valuable tools to tackle this challenge.

Footnotes

1 Further details in "Third normal form," https://en.wikipedia.org/wiki/Third_normal_form

2 Further details in "X/Open XA," https://en.wikipedia.org/wiki/X/Open_XA

3 Full analysis in Jepsen, "Jepsen: Postgres," May 18, 2013, https://aphyr.com/posts/282-jepsen-postgres

4 Full analysis in Kyle Kingsbury, "YugaByte DB 1.1.9," March 26, 2019, https://jepsen.io/analyses/yugabyte-db-1.1.9

5 Full article in Hector Garcaa-Molrna and Kenneth Salem, "Sagas," December 1987, www.cs.cornell.edu/andru/cs711/2002fa/reading/sagas.pdf

6 Book by Bertrand Meyer, "Object-Oriented Software Construction," March 21, 2000, www.amazon.com/gp/product/0136291554

7 Full article in Scott Ambler, "The Object-Relational Impedance Mismatch," www.agiledata.org/essays/impedanceMismatch.html

8 Full article in Greg Young, "CQRS Documents," https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

9 Footnote available in Martin Kleppmann, "Is Kafka a Database?", October 17, 2018, www.confluent.io/kafka-summit-SF18/is-kafka-a-database/

10 Presentation available in Yuri Shkuro, "Conquering Microservices Complexity @Uber with Distributed Tracing," September 4, 2019, www.infoq.com/presentations/uber-microservices-distributed-tracing/

11 See "SOLID," https://en.wikipedia.org/wiki/SOLID

12 Full article in Kirsten Westeinde, “Deconstructing the Monolith: Designing Software that Maximizes Developer Productivity,” February 21, 2019, https://shopify.engineering/deconstructing-monolith-designing-software-maximizes-developer-productivity

13 Interesting article on how to implement it in Jorge Quilcate, “The Importance of Distributed Tracing for Apache Kafka Based Applications,” March 26, 2019, www.confluent.io/blog/importance-of-distributed-tracing-for-apache-kafka-based-applications/

14 Interesting article on how to implement it in Aaron Burk, “Fault Tolerance in Distributed Systems: Tracing with Apache Kafka and Jaeger,” July 24, 2019, www.confluent.io/blog/fault-tolerance-distributed-systems-tracing-with-apache-kafka-jaeger/